言語・オートマトン
有限状態オートマトン(1)

山本章博
国際高等教育院
2026年度版
連絡先: yamamoto.akihiro.5m@kyoto-u.ac.jp



有限状態オートマトンと正則言語

有限状態機械の例(1)

西陣織 Jaccard織機





協力:(有)フクオカ機業

Jaccard織



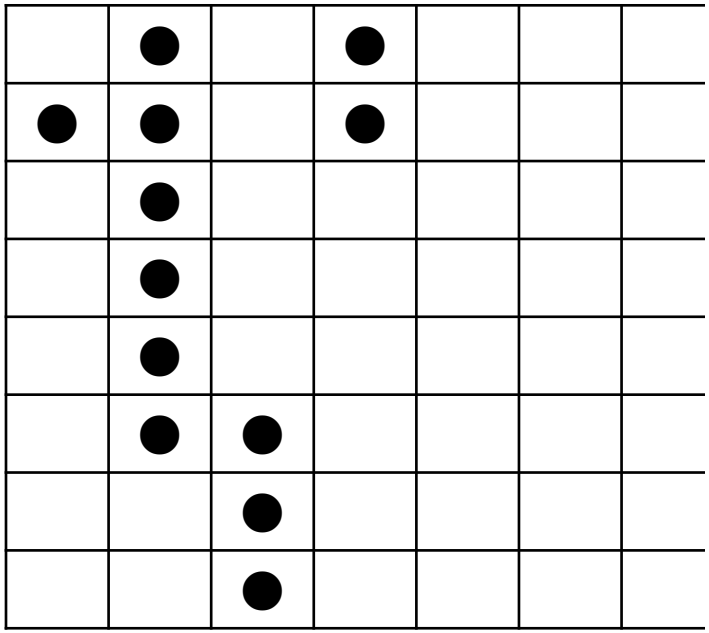






有限状態機械の例(2)

西陣織 Jaccard織機





8bitコードの列とみなす

02 3F E0 03 00 00 00

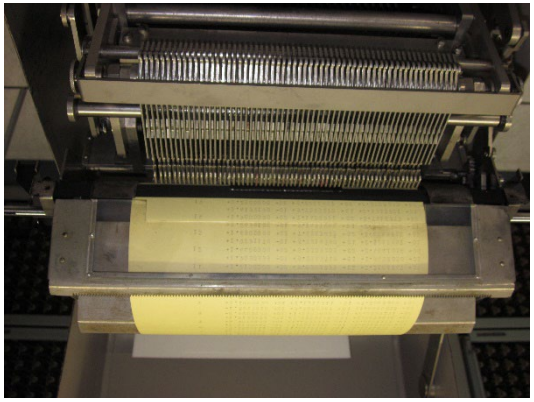
富士通リレー式計算機

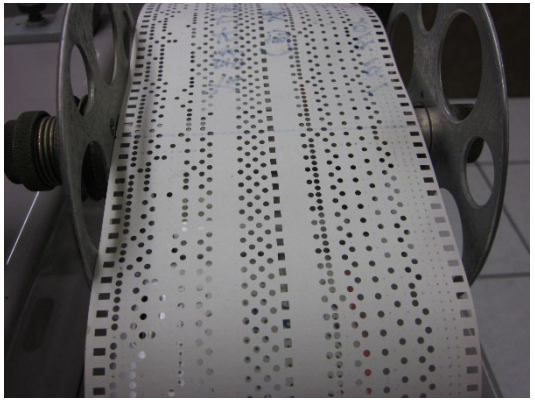
- https://www.youtube.com/watch?v=USPBXiQpxbE
- https://www.youtube.com/watch?v=_j544ELauus







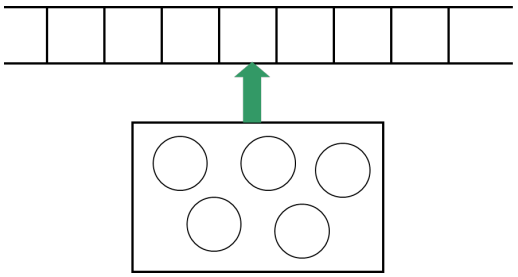




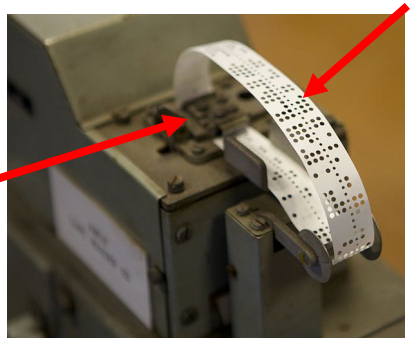


受理機械に共通する性質

- 有限集合であるアルファベット Σ が仮定されている
  - Σの要素である記号(文字)を入力記号と呼ぶ
- マス目に区切られたテープに入力記号を読み書きする
  - マス目１個に1記号(文字)が記入されている
- 受理機械Mには有限個の状態がある.
  - 初期状態(唯一)と受理状態(0個以上)という特別な状態がある
- 受理機械Mの状態は，“現在の状態”と“ヘッドの位置にある記号(文字)”によって他の状態に遷移する
  - 状態遷移は必ず初期状態から始まる



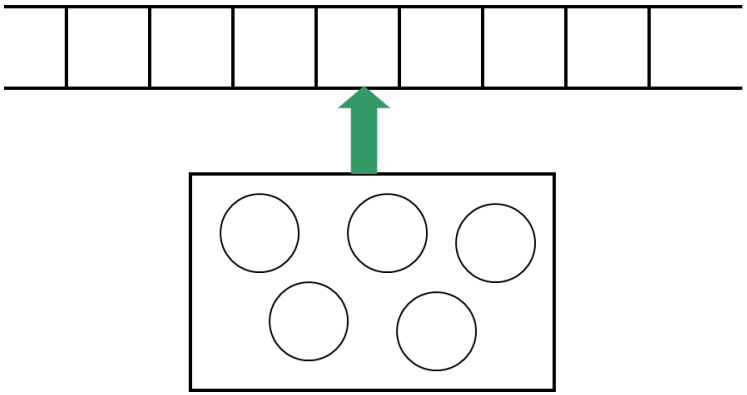
蓋の下にヘッドがあって，
1行(6bit)ずつ読み込む



[Wikipediaより]

有限状態オートマトン

- ヘッドが一方向(右)にしか動かない
- 入力記号列は有限長
  - テープは一方向に無限長（残りは空白で埋められる）
- 記号の書換えを行わない
- 入力記号列を読み終えたときに，
  - 受理状態にあれば，入力記号列は“受理”
  - 受理状態になければ，入力記号列は“不受理”

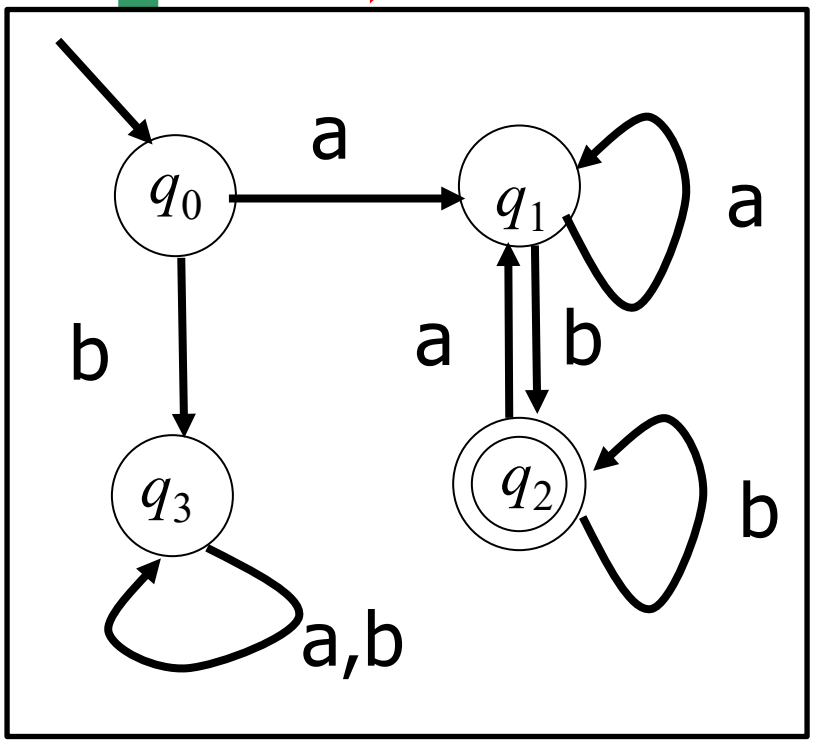


有限状態オートマトンの例(1)

a	b	a	b	a	a	a	b

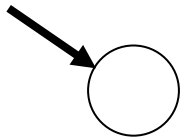
入力アルファベット

$\Sigma = \{ a, b \}$

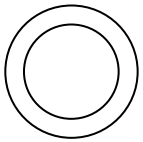


状態遷移図

- ヘッドは必ず左端から右端へ動く.
- 記号を書き換えることはできない.
- 状態遷移は必ず初期状態から始まる.
- 初期状態が決まっている



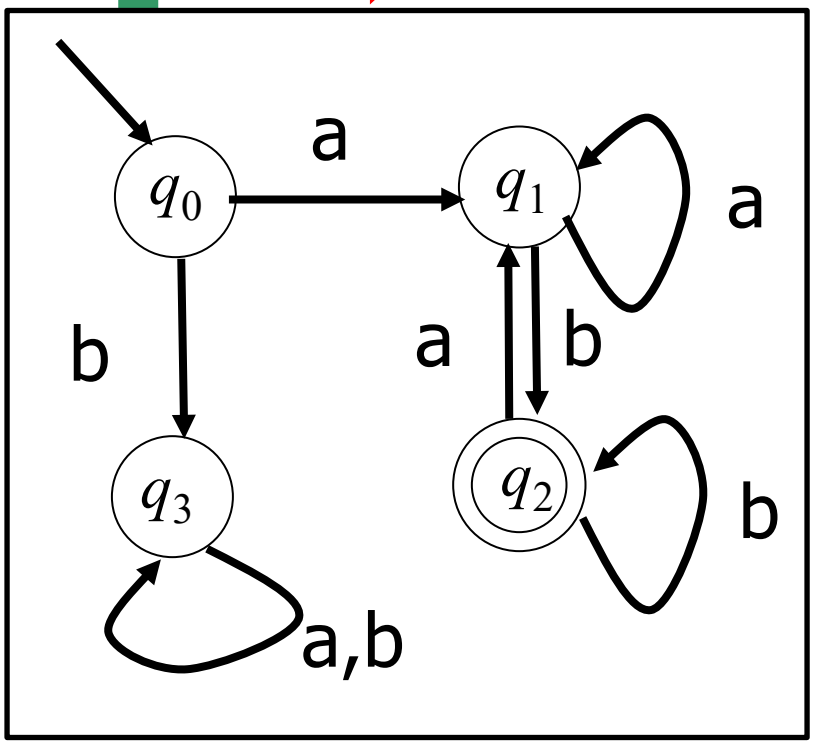
- 受理状態が決まっている(1個とは限らない)



有限状態オートマトンの例(2)

a	b	a	b	a	a	a	b

入力アルファベット
$\Sigma = \{ a, b \}$

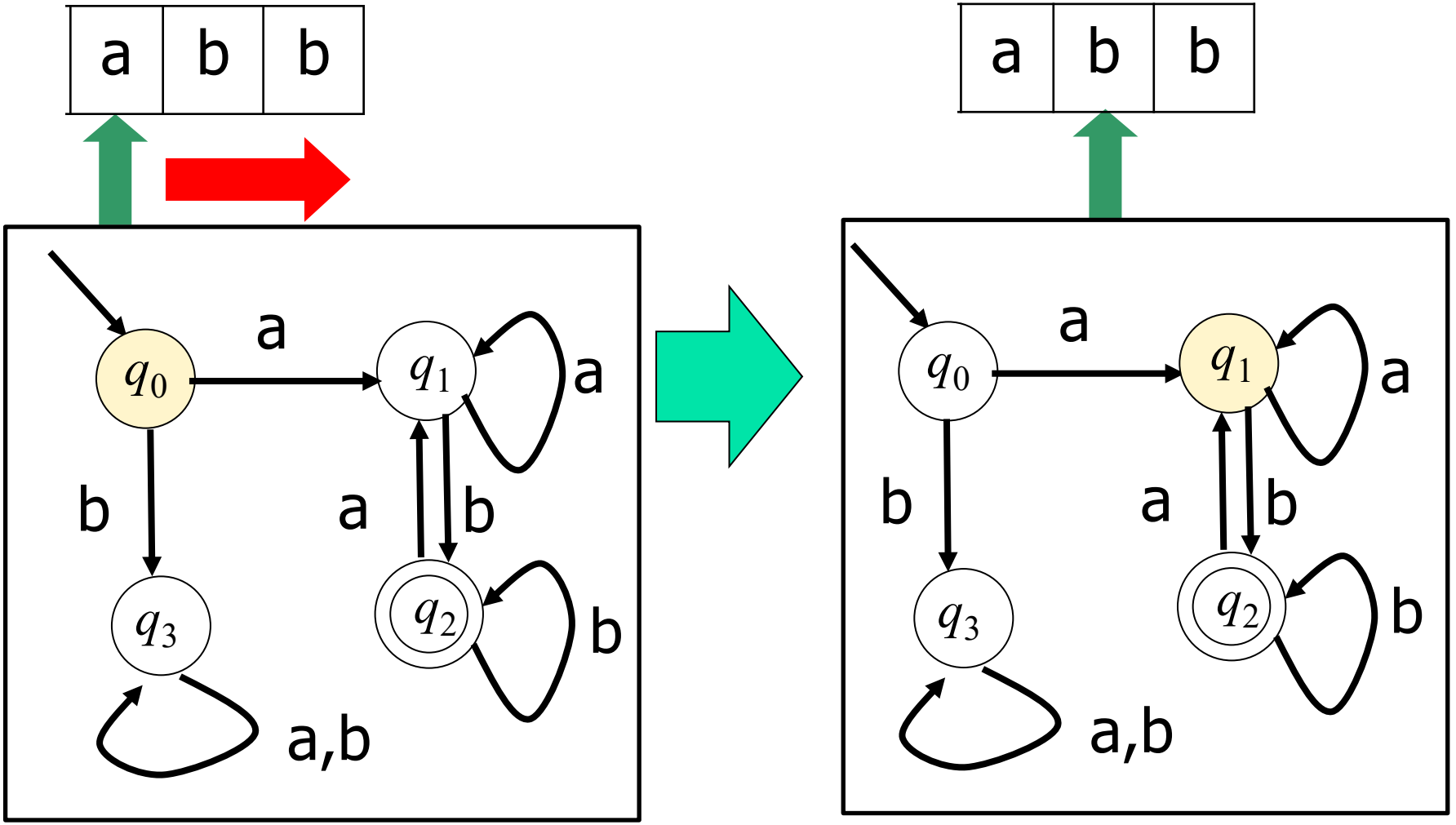


状態遷移図

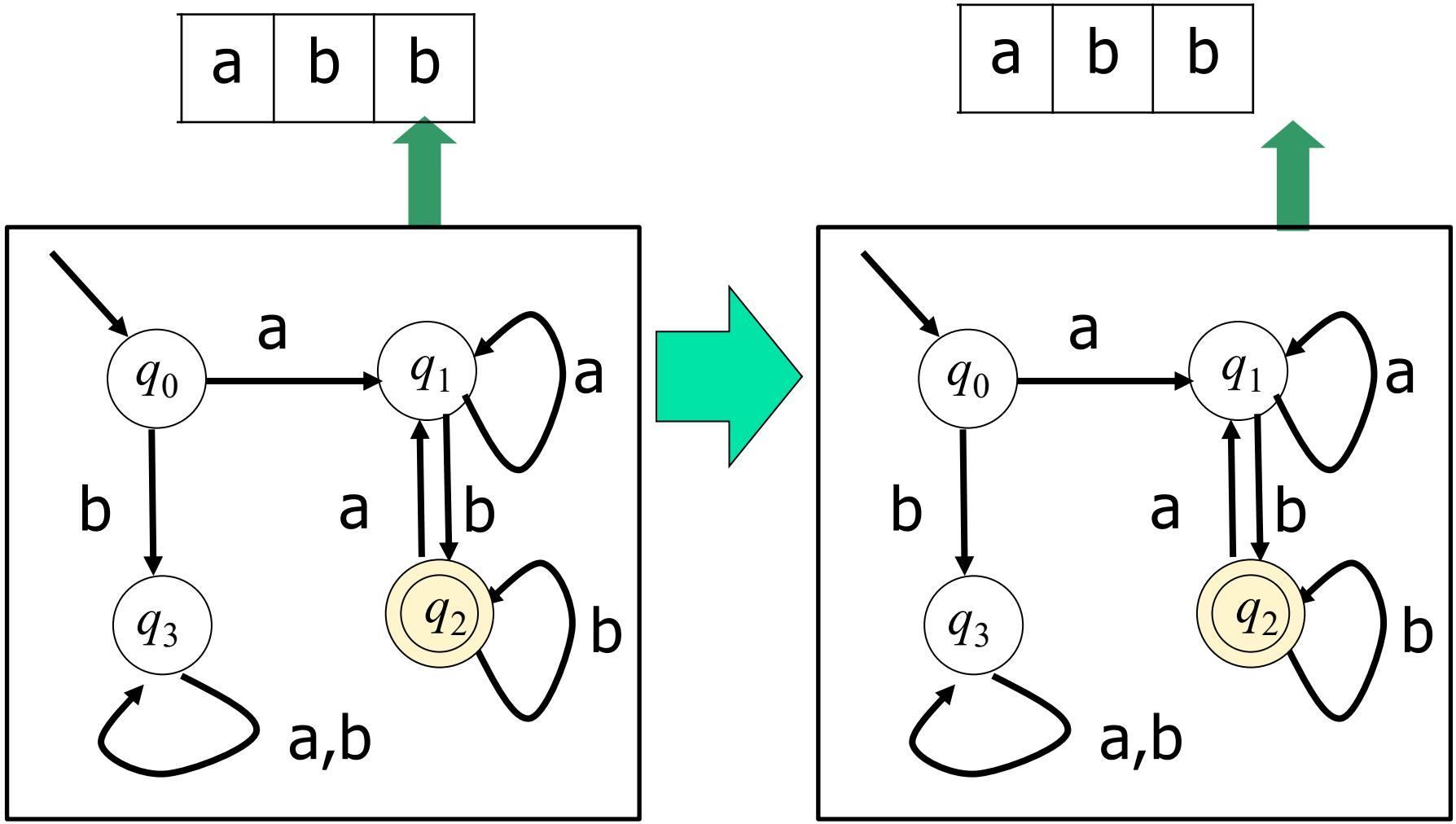
- テープ上の記号をすべて読み終えたとき
  - 受理状態であるとき，テープ上の記号列は受理されるという.
  - 受理状態ではない状態あるとき，テープ上の記号列は受理されないという.

(注) テープ上の記号列は“受理される”か“受理されない”かのいずれかである.

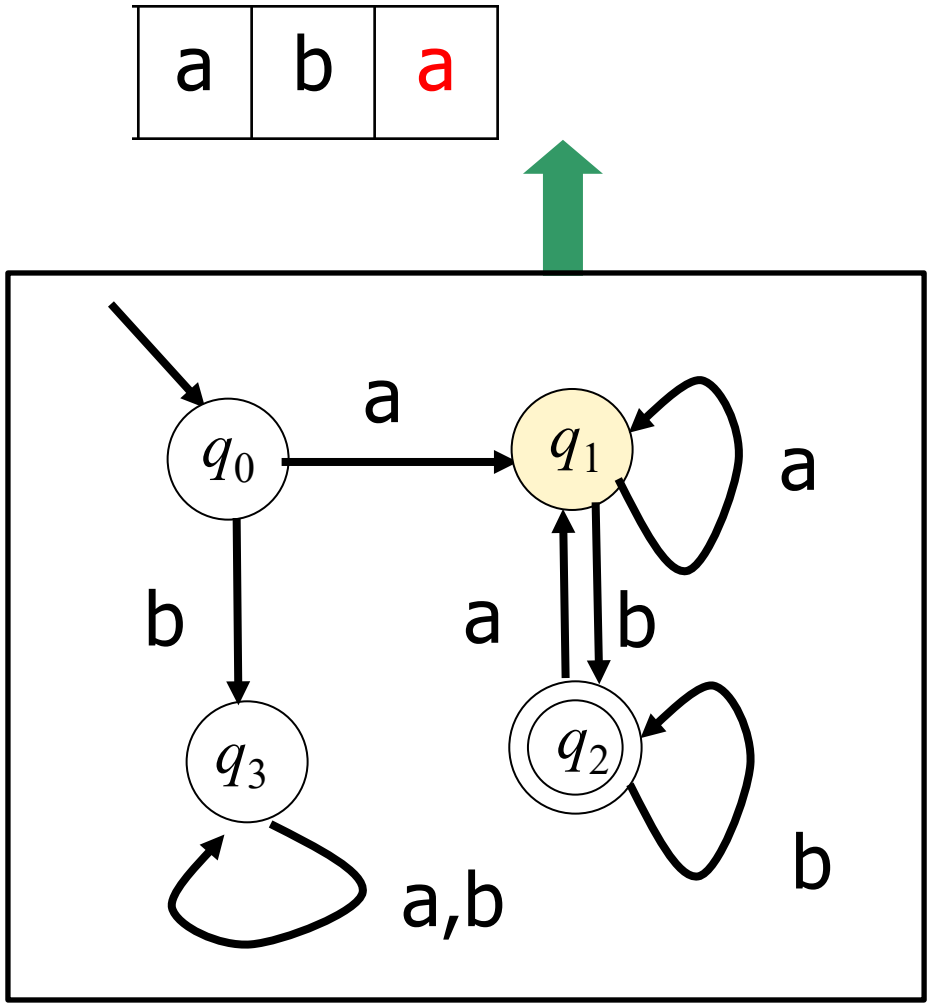
有限状態オートマトンの例(3)



有限状態オートマトンの例(4)



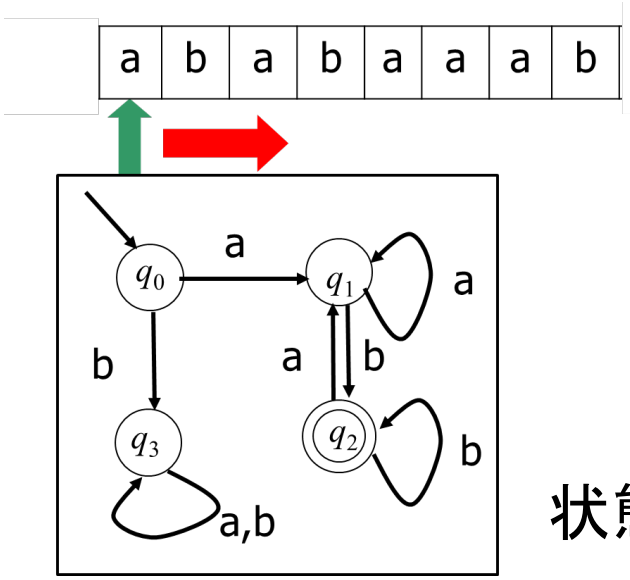
有限状態オートマトンの例(5)



状態遷移図と状態遷移表

入力アルファベット $\Sigma = \{ a, b \}$

	F	a	b
⇒ q_0		q_1	q_3
q_1		q_1	q_2
q_2	○	q_1	q_2
q_3		q_3	q_3



状態遷移図

状態遷移表

- 各行は状態を表す
- 受理状態を表すための列と各入力記号に対応する列がある
- q_i行 c_i列の要素がq_jであれば
  “状態q_iのときヘッドの読む記号がc_jがであれば状態q_kに遷移するを表す

集合と関数による有限状態オートマトンの表現

- 以下のような構成要素からなる組$M=(\Sigma, S, \delta, q_0, F)$を(決定性)有限状態オートマトンとよぶ.
  - Σ は有限アルファベット
    - Σ の要素を入力記号と呼ぶ
  - S は空でない有限集合
    - Sの要素を状態とよぶ.
  - δ は $S\times\ \Sigma \rightarrow S$ なる関数
    - δ は状態遷移表で表現される
  - $q_0 \in S$ であり，初期状態とよぶ.
  - $F \subset S$ であり，Fの要素を受理状態，もしくは終了状態とよぶ.

$\Rightarrow$

	F	c_1	…	c_j	…	c_n
q_0						
…						
q_i				q_k		
…						
q_m						

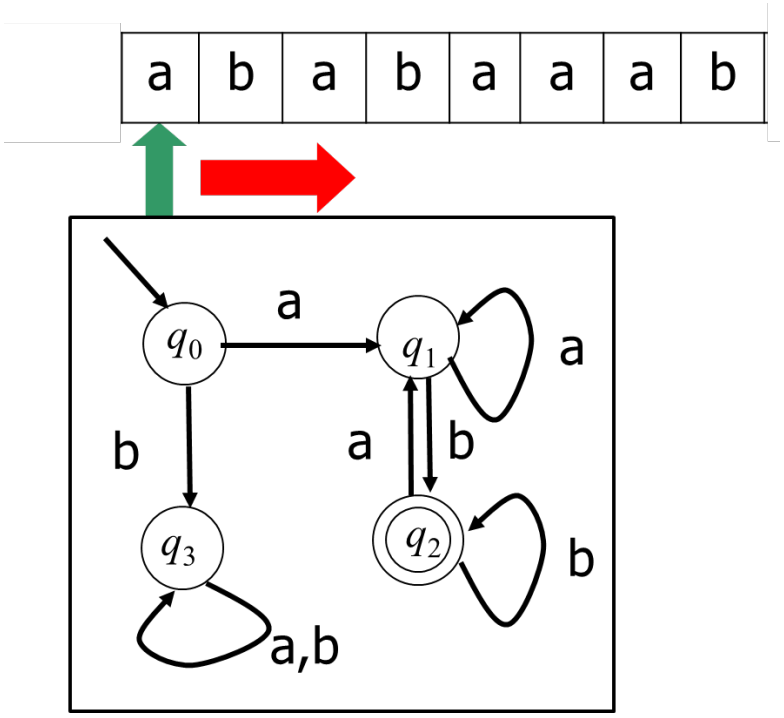
$\delta(q_i, c_j) = q_k$

正則言語(1)

- 有限状態オートマトンMの入力として与える記号列はMによって“受理される”か“受理されない”かのいずれかである.

- 有限状態オートマトンMは言語(Σ^*の部分集合)の内包的な定義に用いることができる.

$$\{ w \in \Sigma^* \mid w \text{ は} M \text{によって受理される} \} = L(M)$$



$L(M) = \{$ab, aab,abb,aaab, aabb,abab,... $\}$

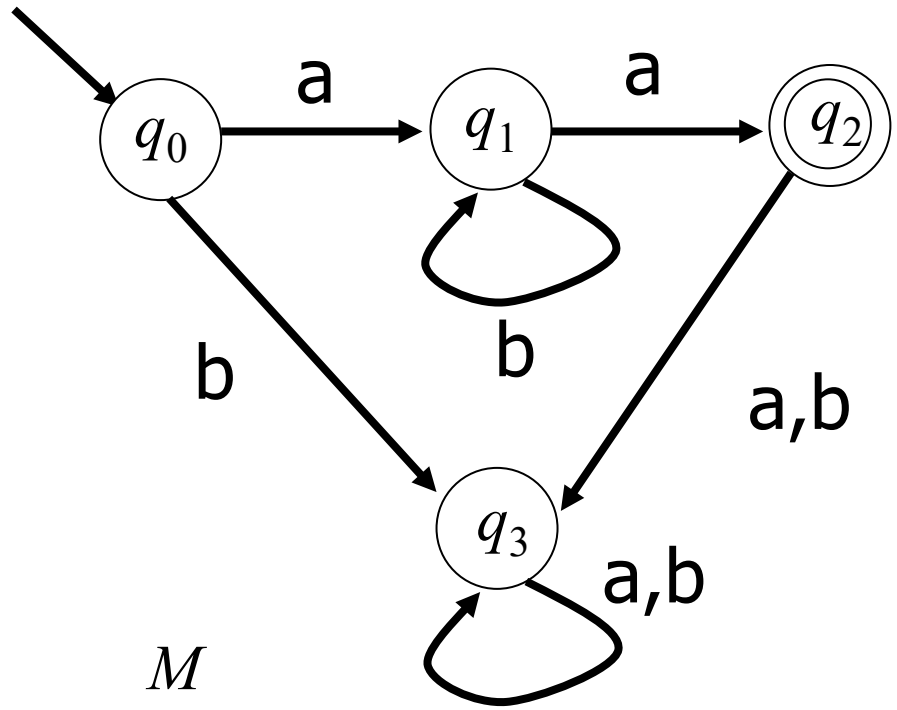
正則言語(2)

- ある言語Lについて$L = L(M)$であるとき,有限状態オートマトンMはLを受理するという.
- 有限状態オートマトンで受理される言語を正則言語(regular language)という.
  - 正規言語という呼ぶことも多い
- “$L = L(M)$であるような有限状態オートマトンMが存在するとき, Lを正則言語という”といういい回しをすることもある.
  - 言語Lが有限状態オートマトン以外の方法で定義されているとき, その定義から$L = L(M)$であるような有限状態オートマトンMを構成することができるか？を問うている.

正則言語(3)

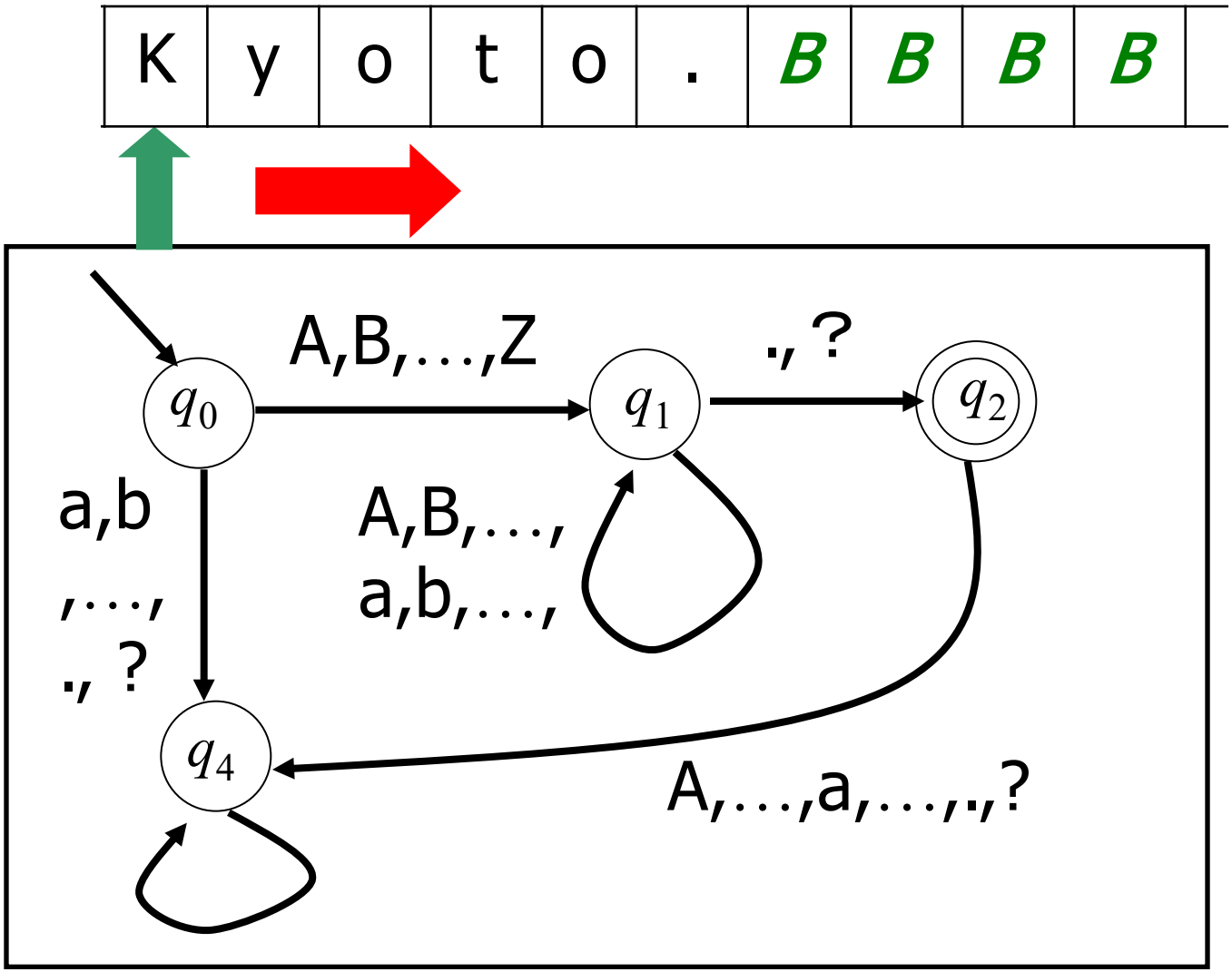
例 入力アルファベットを$\Sigma = \{a, b\}$とする.

言語 $L = \{ab^n a \mid n$ は非負整数$\}$は正則言語である.

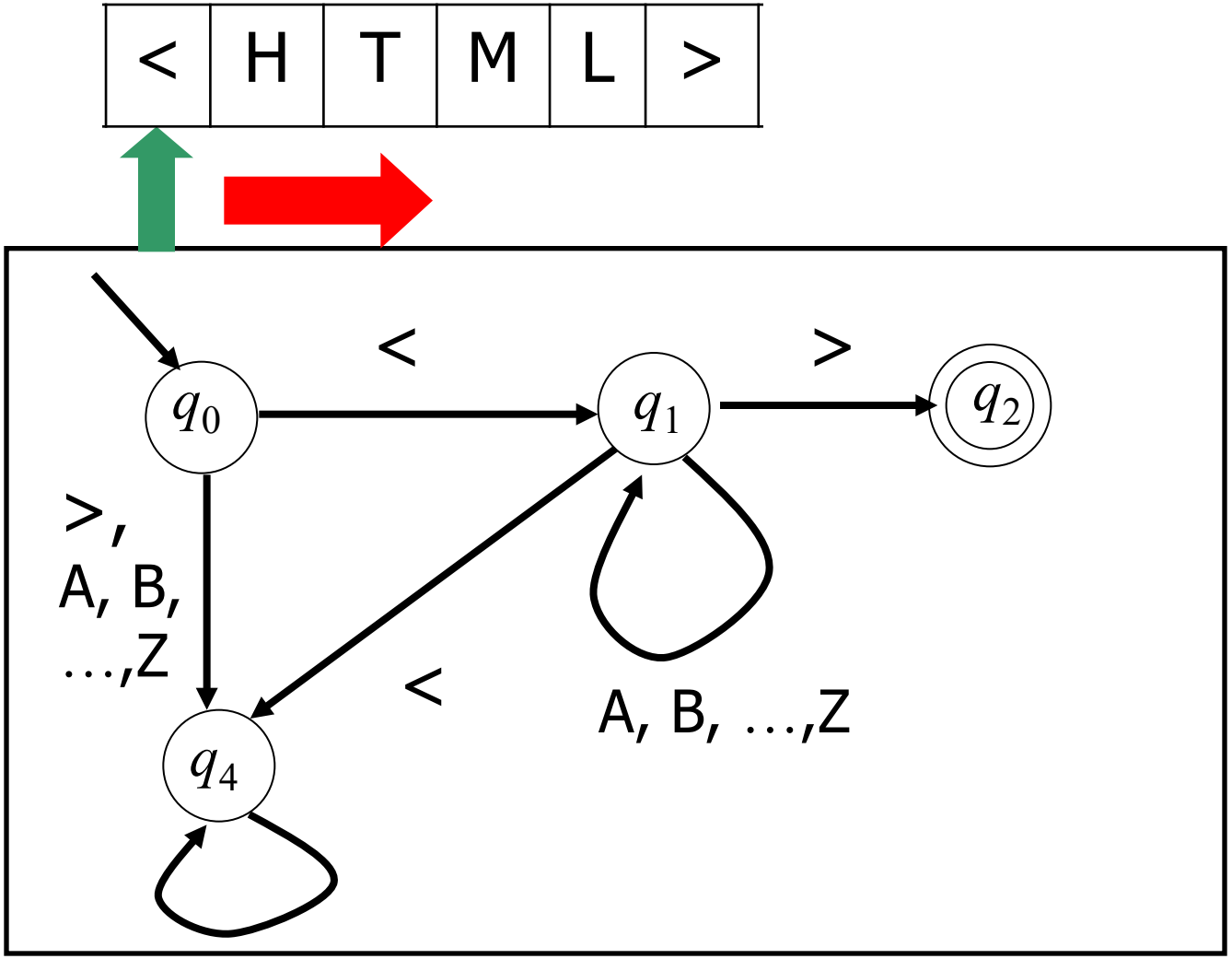


$L \supset L(M)$と$L \subset L(M)$を証明する

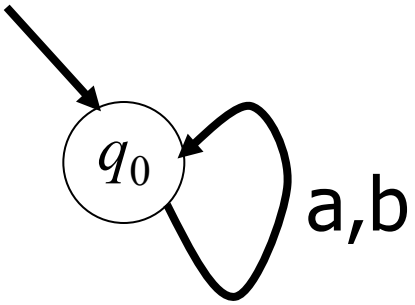
有限状態オートマトン(例)



有限状態オートマトン(例)

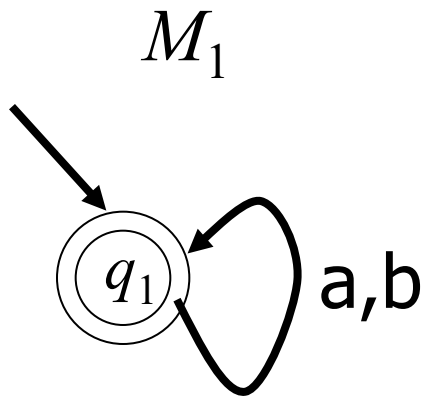


状態数1のオートマトン



	F	a	b
q_0		q_0	q_0

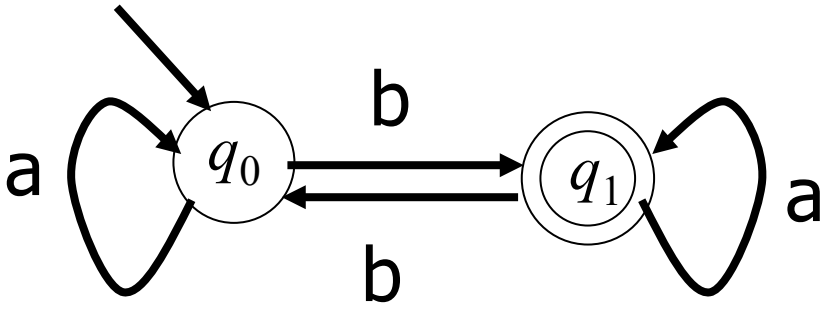
$$L(M_0) = \varnothing$$



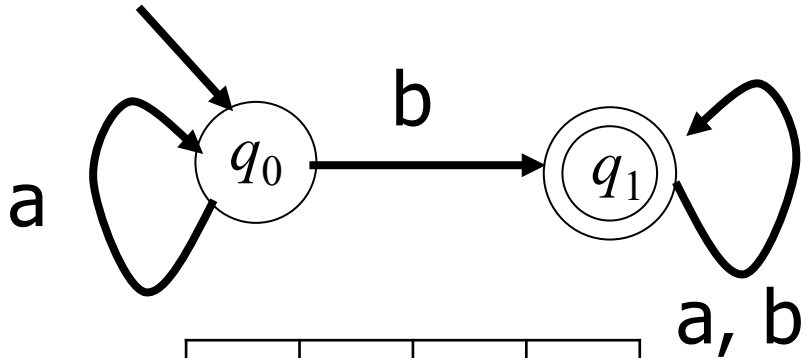
	F	a	b
q_0	○	q_0	q_0

$$L(M_0) = \Sigma^*$$

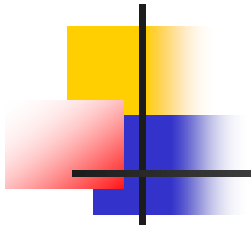
状態数2のオートマトン



	F	a	b
q_0		q_0	q_1
q_1	○	q_1	q_0

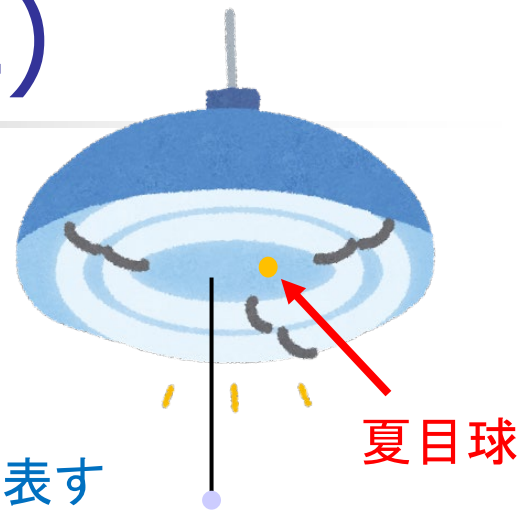


	F	a	b
q_0		q_0	q_1
q_1	○	q_1	q_1

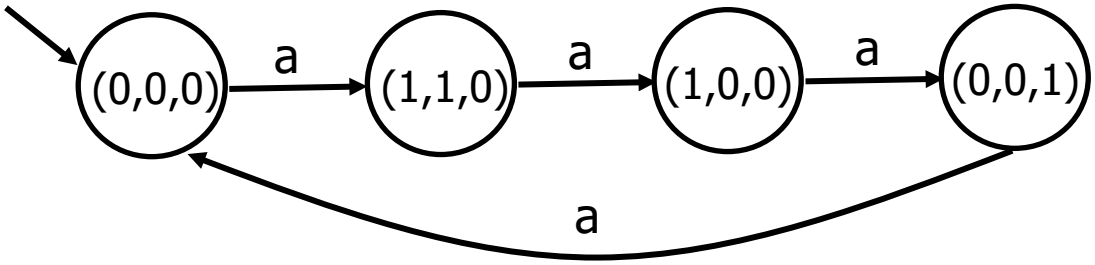


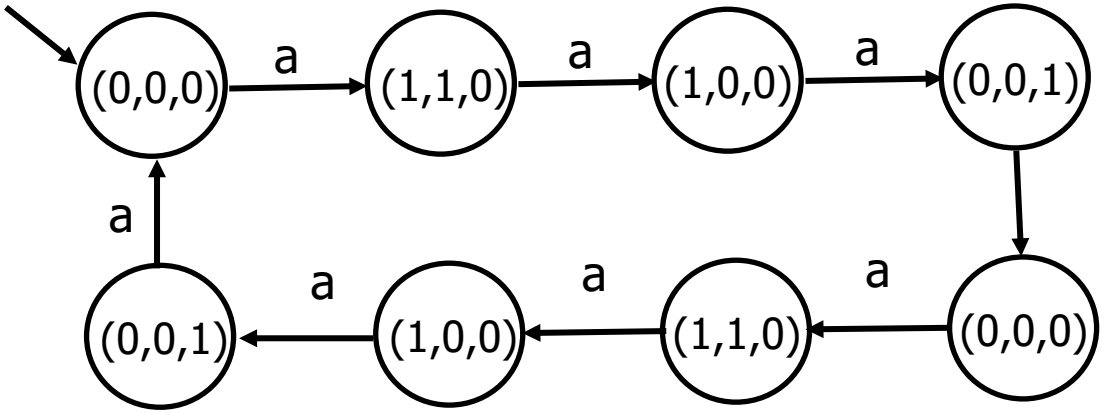
有限状態オートマトンの等価性

有限状態機械の例(2)



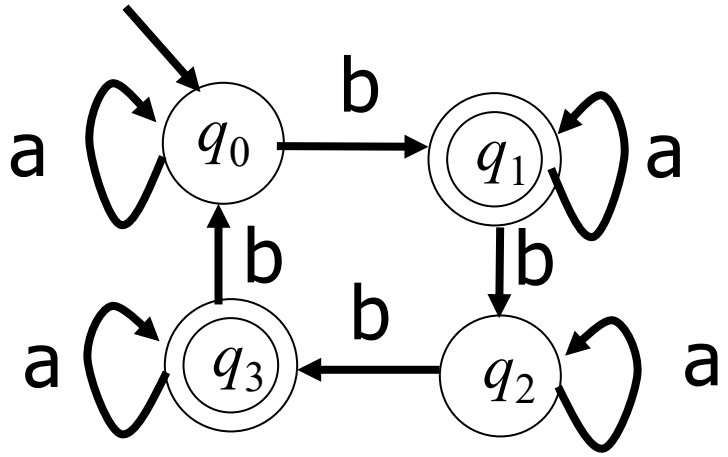
- 内側，外側の2本の蛍光管と
  夏目球から構成されている
- 紐を引くことで操作する：入力記号 a で表す
  - 内側, 外側, 夏目球のON, OFFをブール値の組で表す



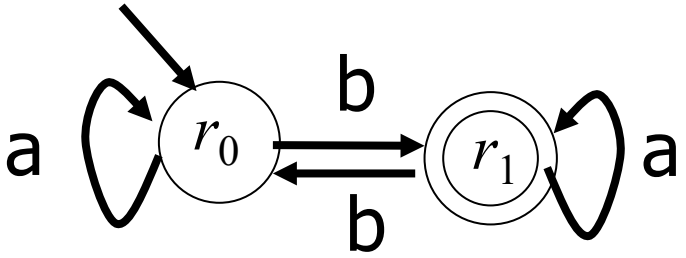


等価なオートマトン

- 下の2つの有限状態オートマトンM_1とM_2が受理する言語は一致する



M_1



M_2

$L(M_1)=L(M_2)$ = {ab, aab, aba, abab, …, abababaaaa,…}

- 有限状態オートマトンM_1とM_2に対して$L(M_1)=L(M_2)$ であるとき, M_1とM_2は等価(equivalent)であるといい, $M_1 \equiv M_2$ と表す.

等価なオートマトンに関する問題

問題1: 有限状態オートマトンM_1とM_2に対して$M_1 \equiv M_2$であるかどうかをどのようにして判定するのか？

- $M_1 \equiv M_2$を判定する“アルゴリズム”を与えよ.

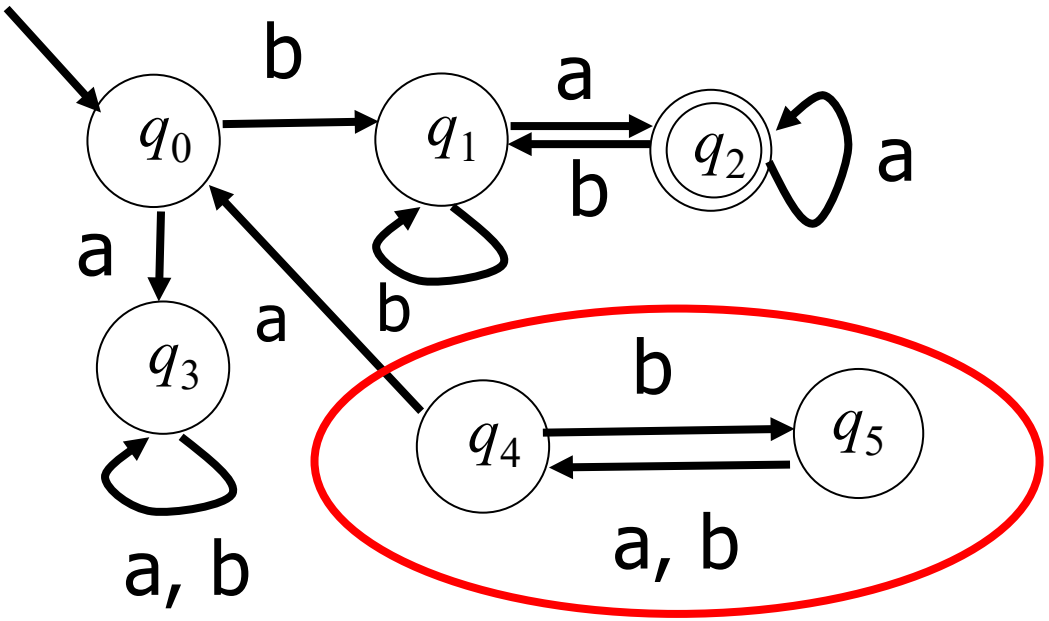
問題2: 有限状態オートマトンMに対して$M \equiv M'$を満たすM'で，状態数が最小のものを求めよ.

- 数学的にはこのようなM'が存在することはわかる
- このようなM'は唯一なのか？複数個あるのか？
- M'を求める“アルゴリズム”を与えよ.

到達不可能な状態

- 下図の有限状態オートマトンMにおいて，状態q_4, q_5は初期状態からどのような状態遷移によっても到達できない
- Mからq_4, q_5を除いて得られる有限状態オートマトンM'については

$$M \equiv M'$$



状態数以上の語に対する動作

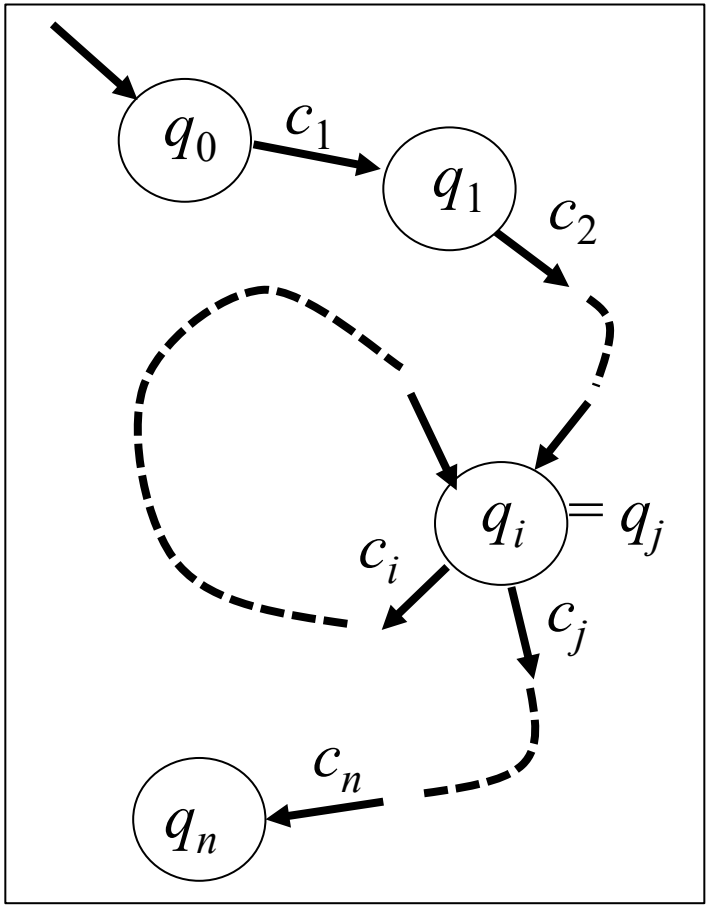
補題 有限状態オートマトンMの状態数をnとするとき, 長さnの語$w = c_1c_2\ldots c_n$をMの入力として与えて

$$q_0 \overset{c_1}{\Rightarrow} q_1 \overset{c_2}{\Rightarrow} q_2 \overset{c_3}{\Rightarrow} \cdots \overset{c_n}{\Rightarrow} q_n$$

と遷移したとすると, $q_i = q_j$ $(i \neq j)$ なる組が存在する

- 鳩ノ巣原理による.

$\delta(q_0, c_1) = q_1, \delta(q_1, c_2) = q_2, \ldots$ という意味



系 補題においてwの部分語w'で$|w'|<|w|$かつ, 一度も同じ状態に遷移せずにq_nに到達するものを構成することができる.

- 系から, q_0から到達可能な状態を列挙するには, 長さ$n-1$以下の語wを網羅的に入力すればよい.

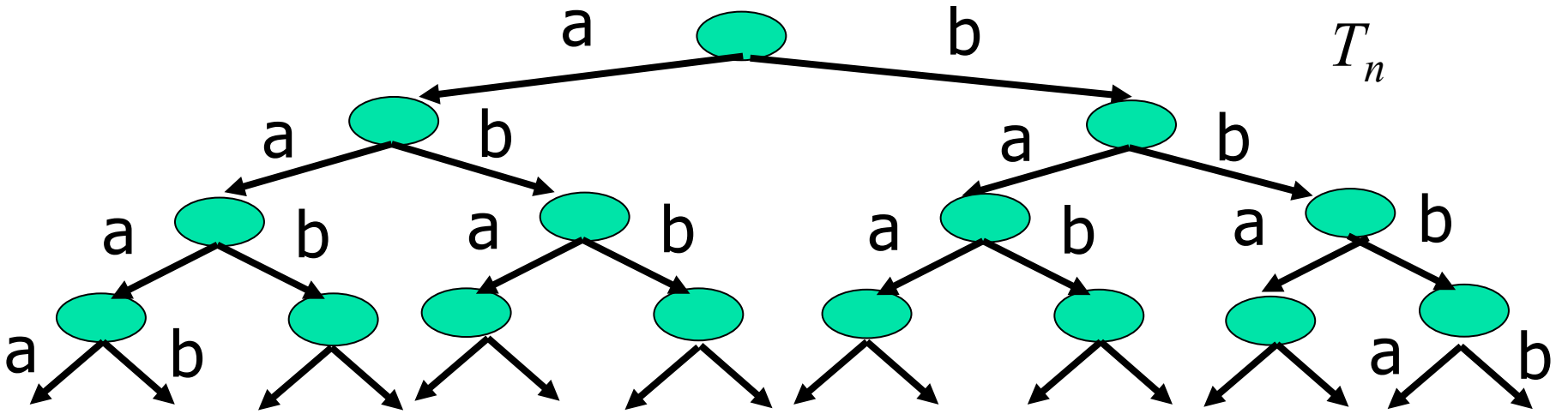
オートマトンの等価性判定の準備

- 長さ n+1以下のすべての記号列の集合 $\Sigma^*|_{\leq n-1}$

例 $\Sigma = \{a, b\}$

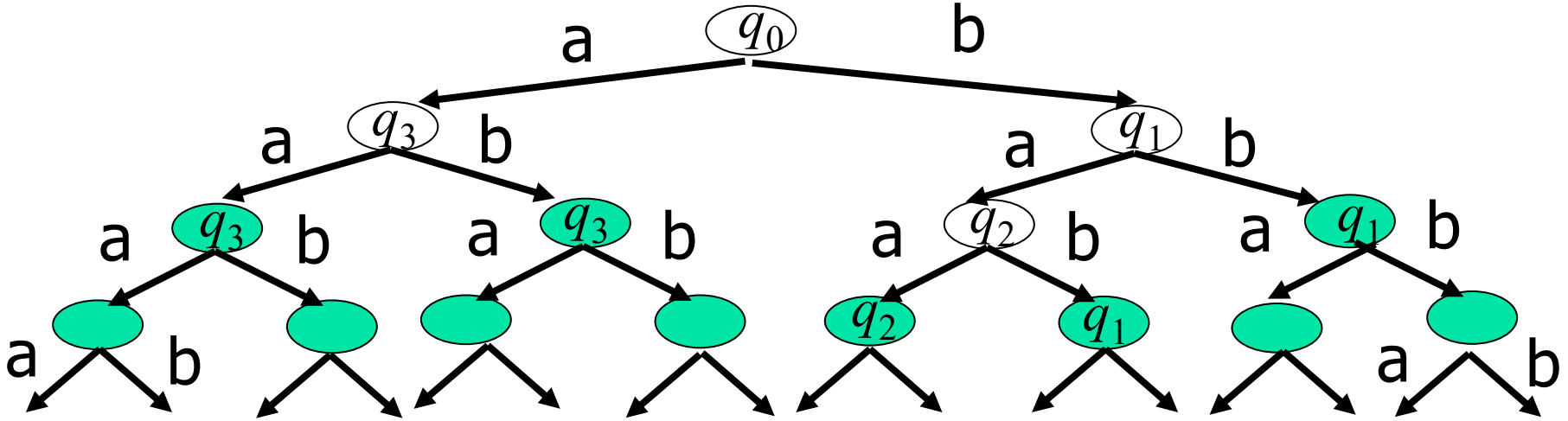
$\Sigma^*|_{\leq n-1} = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \ldots, a^{n-1}, \ldots, b^{n-1}\}$

- $\Sigma^*|_{\leq n-1}$の要素の構成方法は高さn +1の木構造T_nで表現できる.
  - $\Sigma^*|_{\leq n-1}$の前綴木(prefix tree)という.



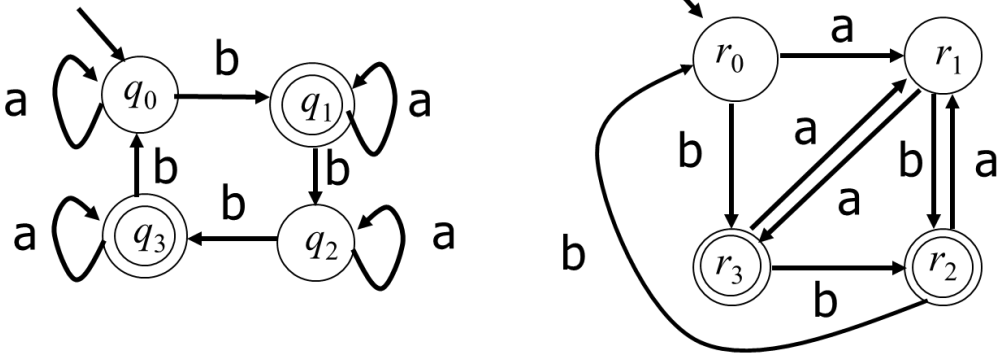
到達不可能な状態の検出

- T_nの根から順に節点(ノード)にMの状態をラベル付けする
  - $n = M$の状態数
  - 同じ状態が出てきたら，それ以上操作を続ける必要はない



- 前綴木T_nの根から葉に至る各経路の長さはMの状態数nより大きいから鳩ノ巣原理より，各経路には必ず2度以上ラベル付けされる状態が存在する.
- T_nのラベル付けされた節点からなる部分木に，ラベルとして出現しない状態が到達不可能な状態である.

等価性判定(1)

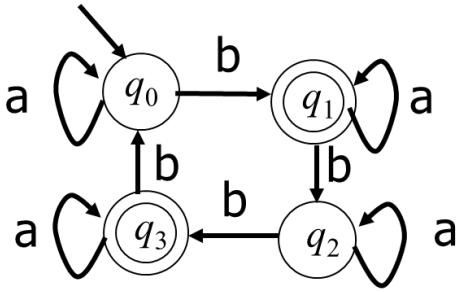


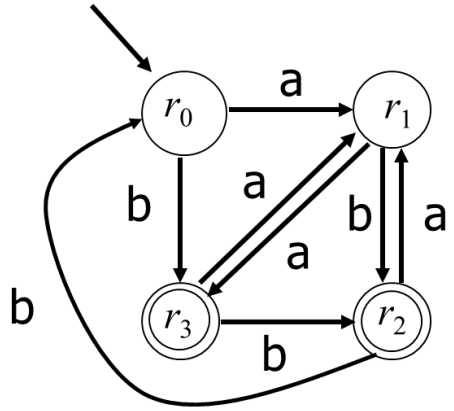
方針

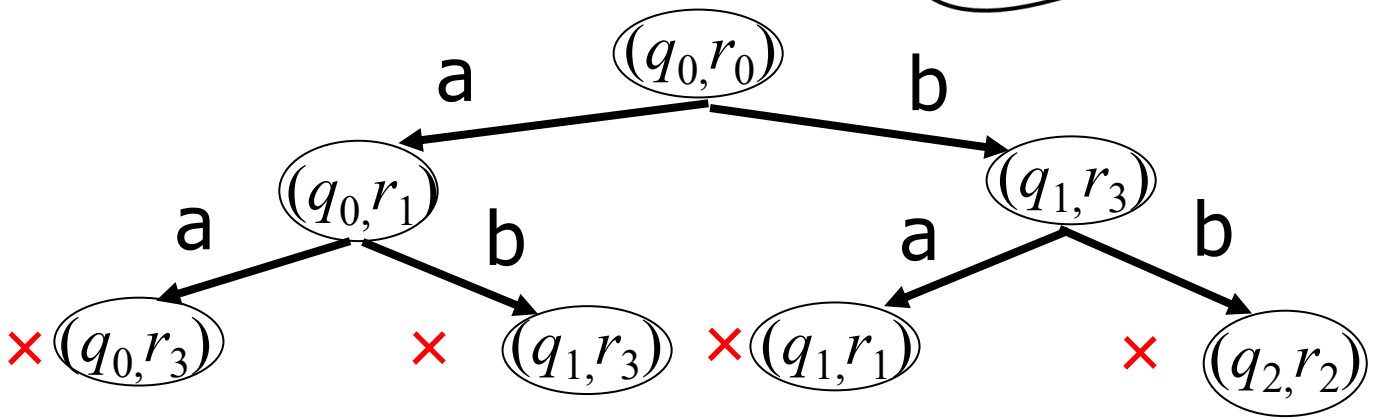
- M_1の状態数n_1, M_2の状態数n_2, $n = n_1 n_2$
- 長さ n 以下の記号列 w を受理するかどうかを網羅的に調べる
- 受理すると非受理状態の組が出てきたら終了

等価性判定(1)

- M_1の状態数n_1, M_2の状態数n_2, $n = n_1 n_2$
- 前綴木T_{n+1}の節点(ノード)にM_1の状態とM_2の状態の組をラベル付けする
- 受理状態と非受理状態の組が出てきたら終了

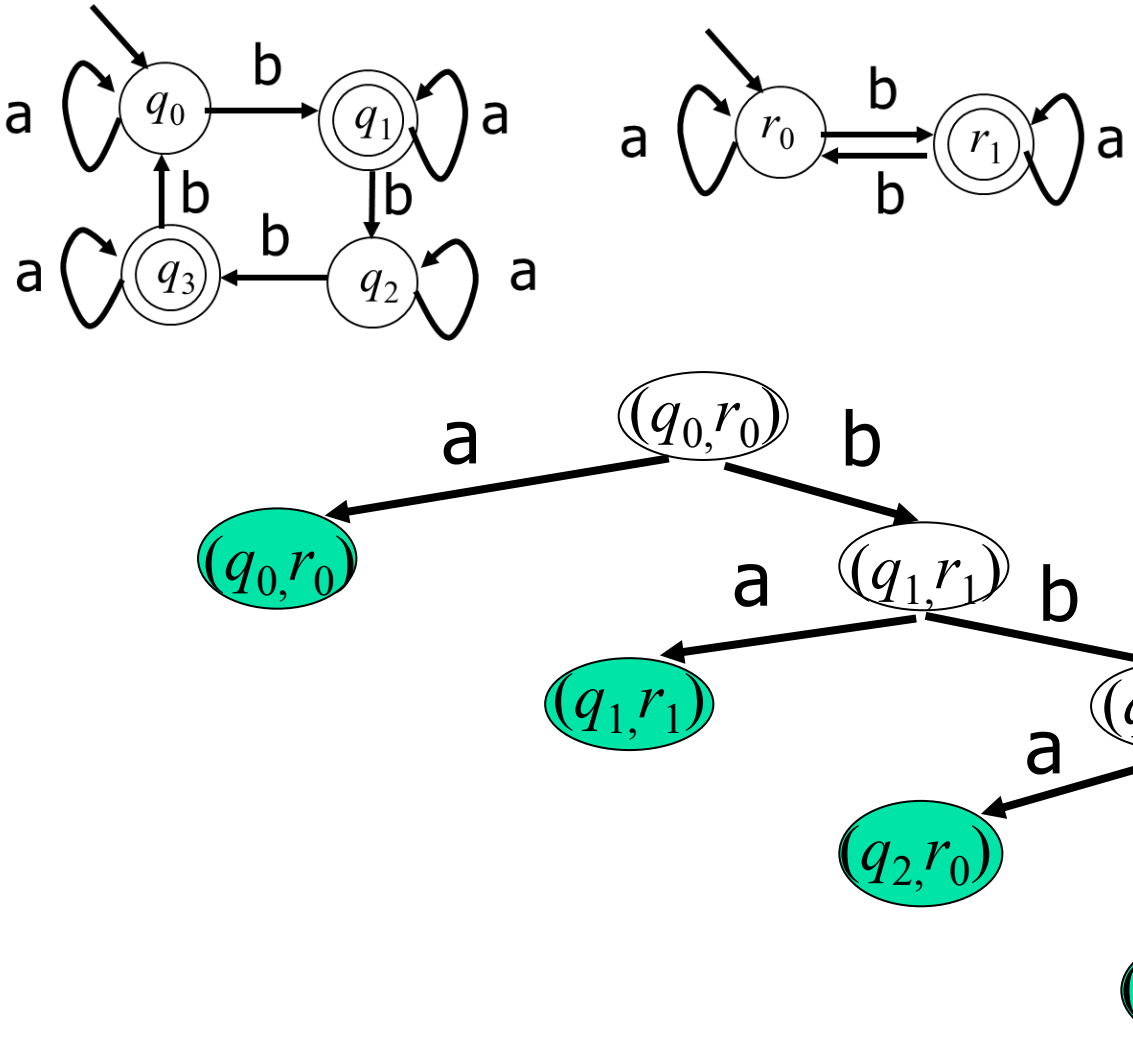






等価性判定(2)

- 同じ状態の組が出てきたら，それ以上操作を続ける必要はない



等価性判定(2)の根拠

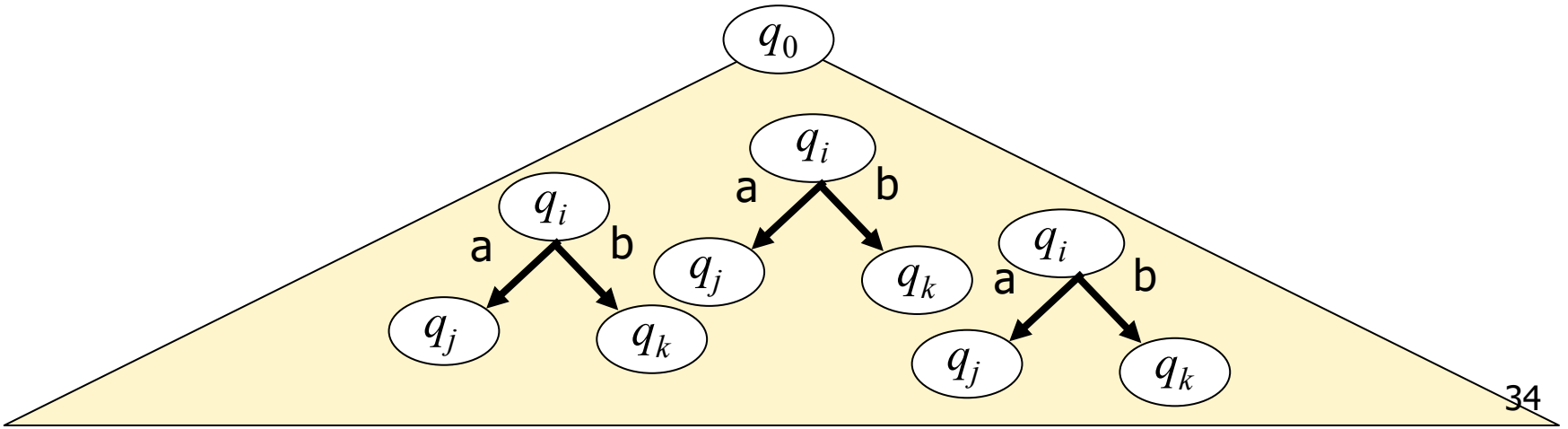
- 有限状態オートマトンの動作は，状態qと記号cだけで決まる

  $\delta: S \times \Sigma \rightarrow S$, $\delta(q, c)=q'$

  - 状態qと記号c以外に遷移先を変える方法はない



- 前綴木の異なる節点に同じ状態q_iをラベル付けすれば，子の節点のラベルはすべて同じ



等価性判定(2)の根拠

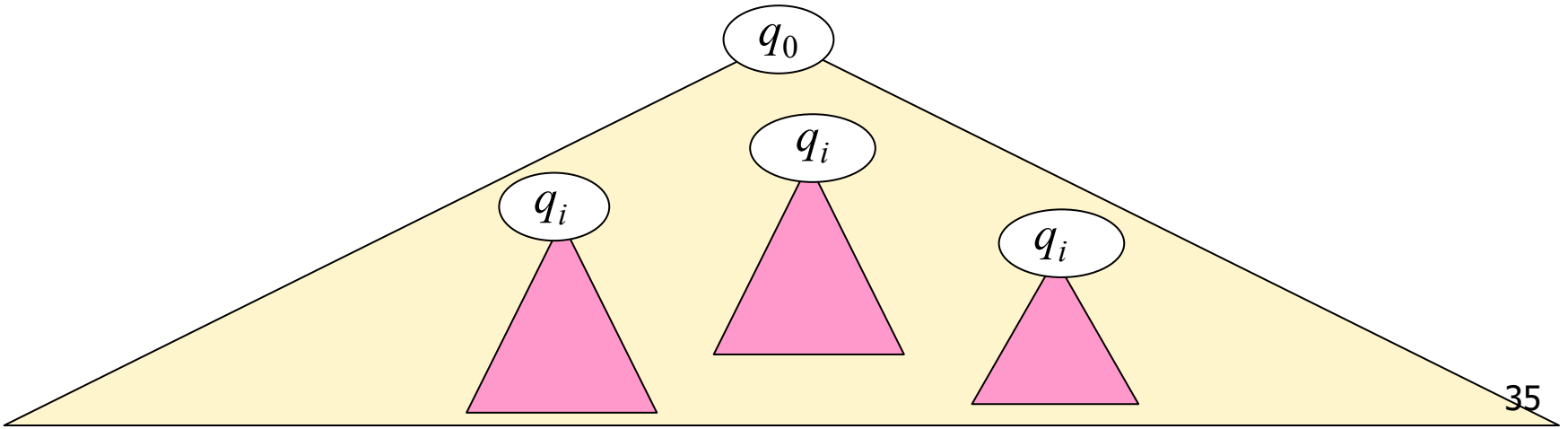
- 有限状態オートマトンの動作は，状態qと記号cだけで決まる

  $\delta: S \times \Sigma \rightarrow S$, $\delta(q, c)=q'$

  - 状態qと記号c以外に遷移先を変える方法はない

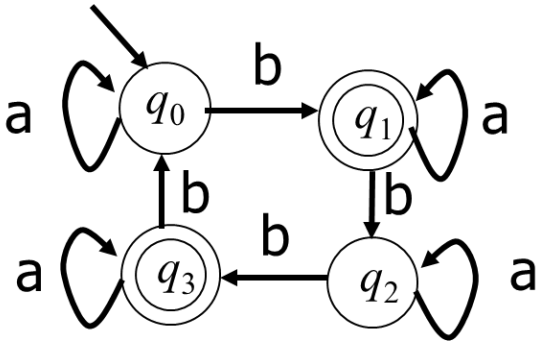


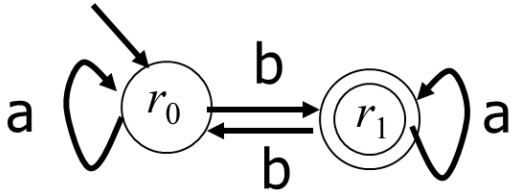
- 前綴木の異なる節点に同じ状態q_iをラベル付けすれば，子の節点のラベルはすべて同じ
- ラベルが状態q_iであるような節点を根とする部分木の節点のラベルは，任意の深さまで同じ

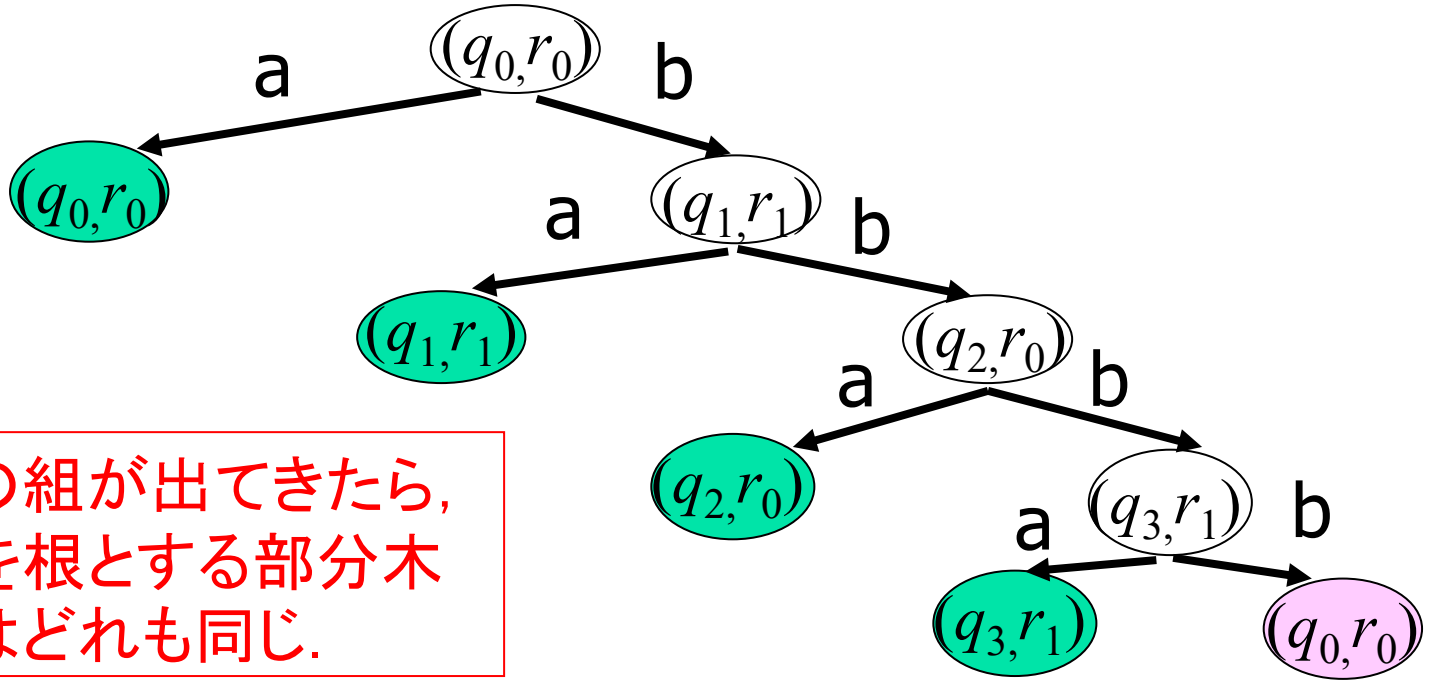


等価性判定(2)

- 同じ状態の組が出てきたら，それ以上操作を続ける必要はない







同じ状態の組が出てきたら，
その節点を根とする部分木
のラベルはどれも同じ．

状態数以上の語に対する動作

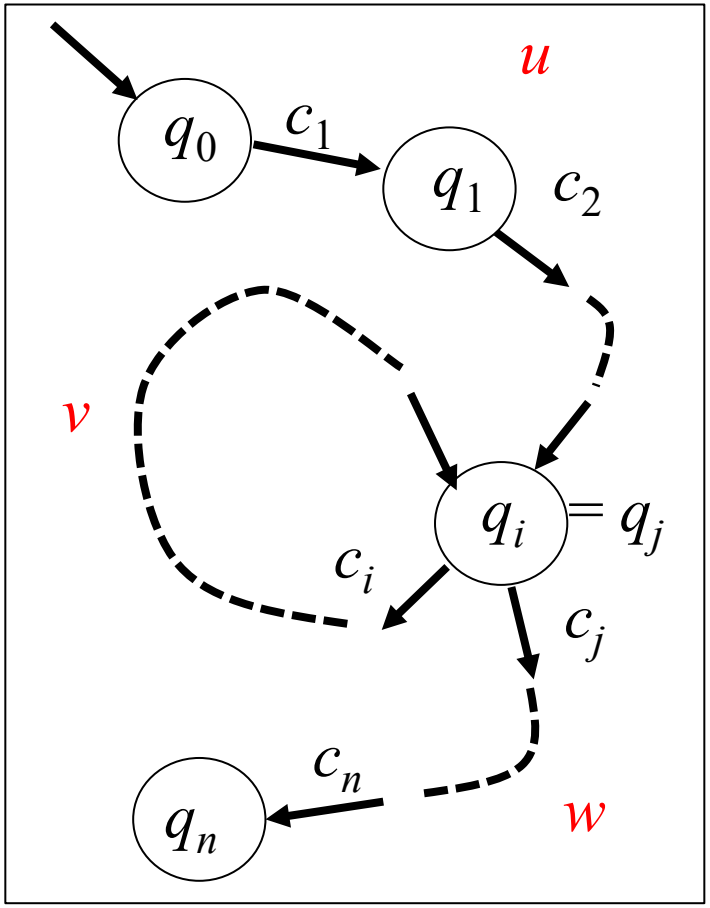
補題 有限状態オートマトンMの状態数をnとするとき, 長さnの語$s = c_1c_2\ldots c_n$をMの入力として与えて

$$q_0 \overset{c_1}{\Rightarrow} q_1 \overset{c_2}{\Rightarrow} q_2 \overset{c_3}{\Rightarrow} \ldots \overset{c_n}{\Rightarrow} q_n$$

と遷移したとすると, $q_i = q_j$ $(i \neq j)$ なる組が存在する

- 鳩ノ巣原理による.

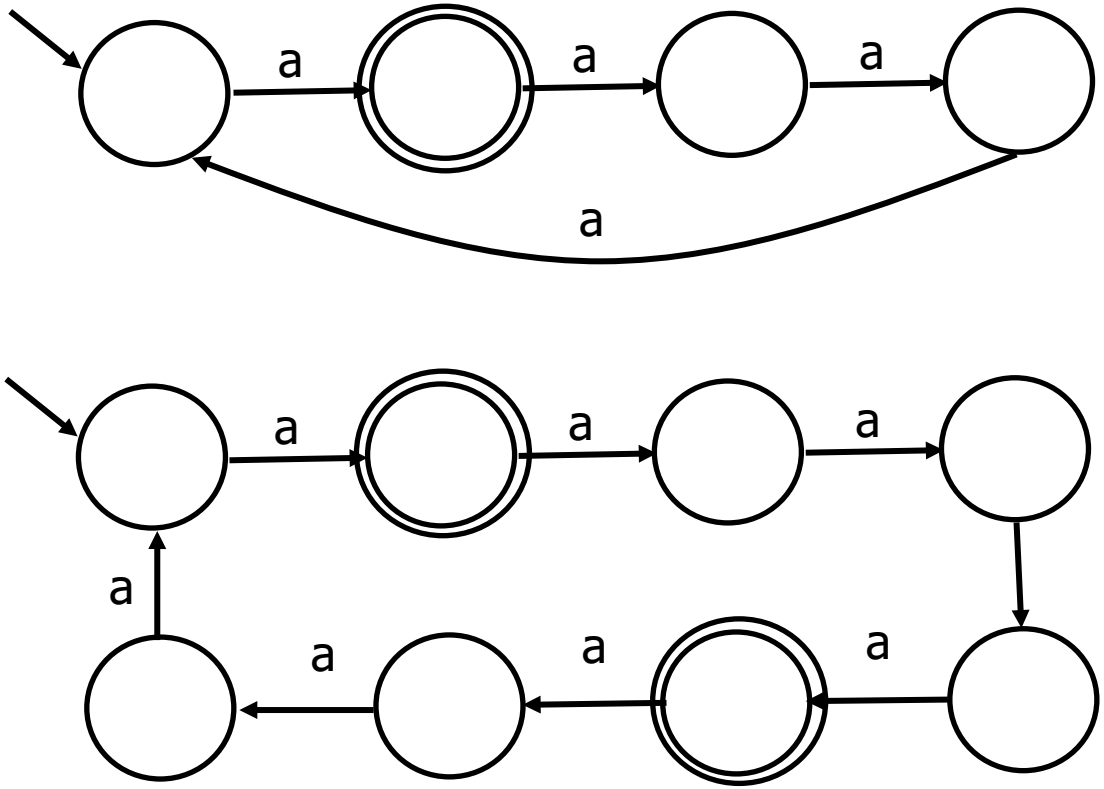
$\delta(q_0, c_1) = q_1, \delta(q_1, c_2) = q_2, \ldots$ という意味



系 補題においてsの部分語u, wで$s=uvw$かつ$|uw|<|s|$かつ, 一度も同じ状態に遷移せずにq_nに到達するものを構成することができる.

有限状態機械の例(1)より

- $L=\{ a^n \mid n = 4m+1, m \in \mathbf{N}\}$を受理する有限状態オートマトン

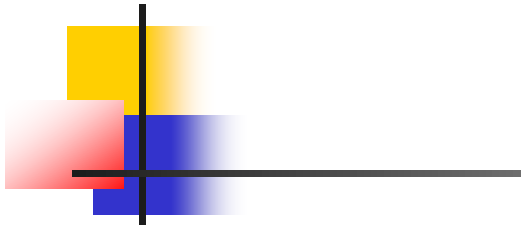


正則言語の特徴

Pumping Lemma (要素数が有限ではない)正則言語Lに対して, 自然数Nがあって, $s \in L$かつ$|s| \geq N$ なるすべての語sは $s = uvw$ かつ $|v| \geq 1$かつ すべての$i \geq 0$に対して $uv^iw \in L$ が成り立つような部分語vを含んでいる.

- NはLに依存して定まる(Lが変わると Nは変わる)
- Nは$L=L(M)$なる有限状態オートマトン Mの状態数とすればよい.

(注) 自然数の除法の基本式 $b = qa+r$ と類似している



有限状態オートマトンの状態数最小化

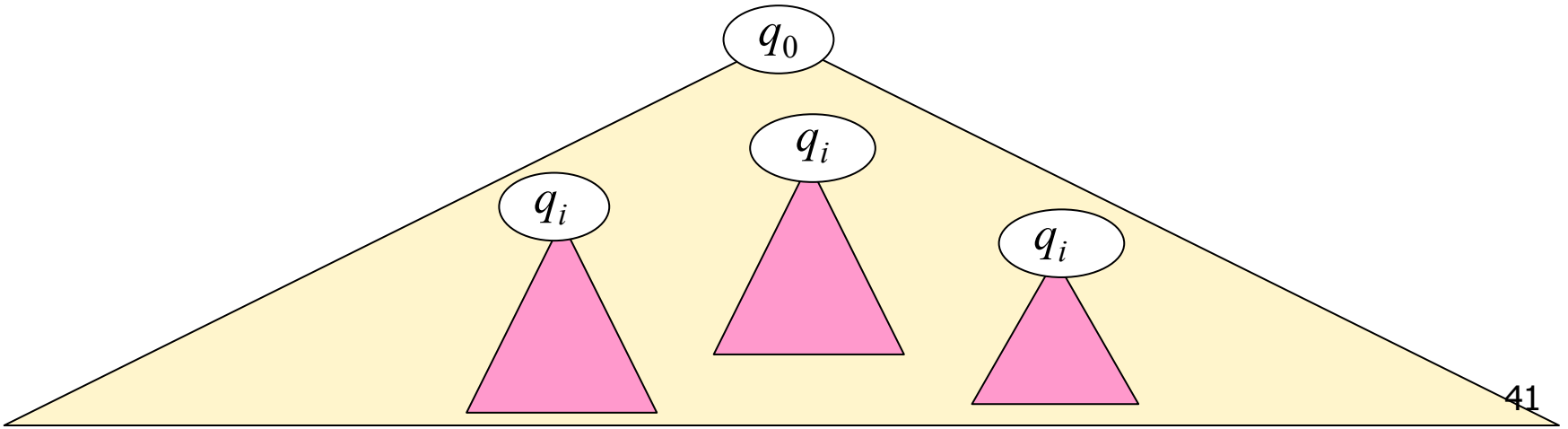
等価性判定(2)の根拠より

- 有限状態オートマトンの動作は，状態qと記号cだけで決まる

  $\delta: S \times \Sigma \rightarrow S$ ， $\delta(q, c)=q'$

  - 状態qと記号c以外に遷移先を変える方法はない

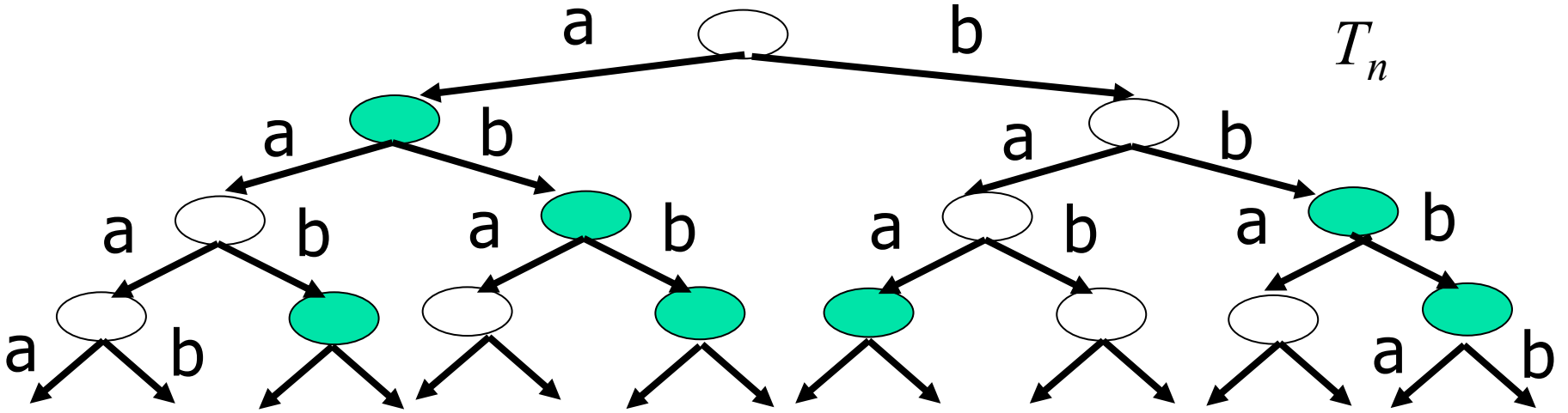
- 前綴木の異なる節点に同じ状態q_iをラベル付けすれば，子の節点が受理状態か否かはすべて同じ
- ラベルが状態q_iであるような節点を根とする部分木の節点が受理状態か否かは任意の深さまで同じ



オートマトンの最小化の準備(1)

有限状態オートマトン M の状態数：n

- 前綴木(prefix tree)T_nの節点の総数は$K = |\Sigma|^n - 1$
  - 各節点に受理状態か否かをラベル付けする方法は 2^K
- ある節点Nから長さ $n-1$の記号列を入力すれば，その節点にラベル付けされるはずだった状態は出尽くしている．



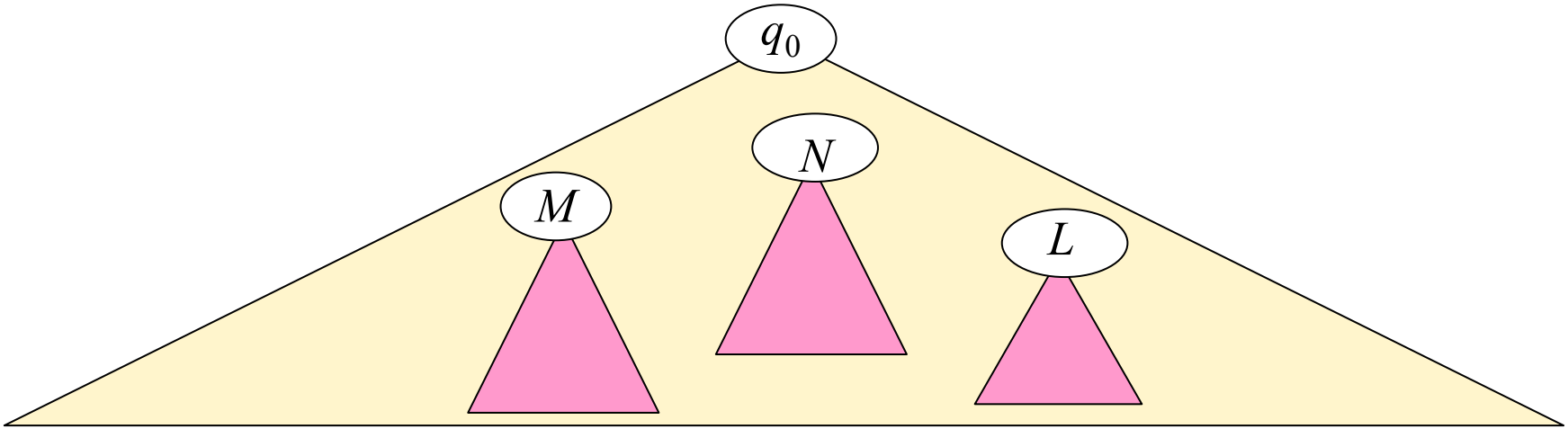
オートマトンの最小化の準備(2)

有限状態オートマトン M の状態数：n

- ある節点Nから長さ$n-1$の記号列を入力すれば，その節点にラベル付けされるはずだった状態は出尽くしている．

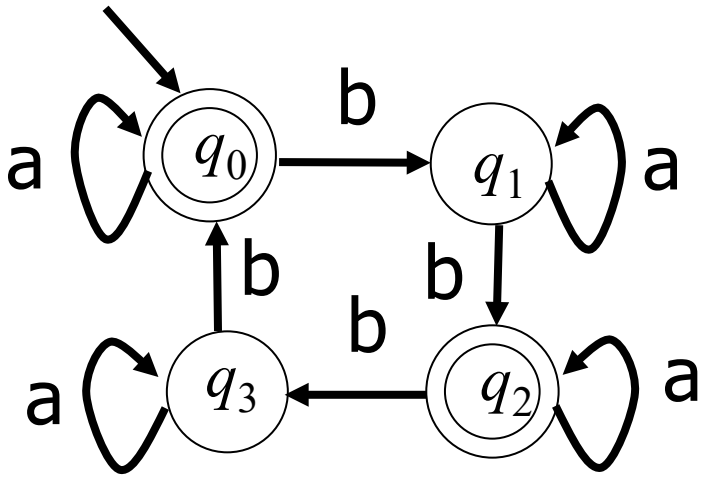


- 節点Nを根とする高さnの部分木と節点Mを根とする高さnの部分木のラベル付けが全く同じであれば，両者を区別する必要はない．

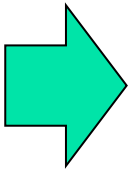


状態数最小化(step 1)

- 状態を終了状態と非終了状態にだけ分類して，状態遷移表を書き換え
- 各行のパターンを分類



	F	a	b
q_0	○	q_0	q_1
q_1		q_1	q_2
q_2	○	q_2	q_3
q_3		q_3	q_0

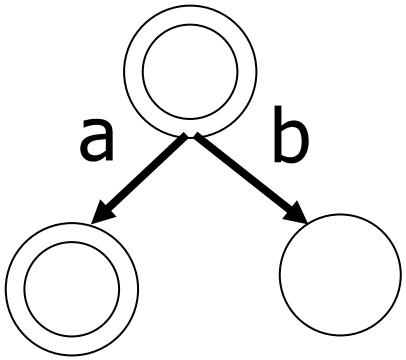


	F	a	b	
q_0	1	1	0	A
q_1	0	0	1	B
q_2	1	1	0	A
q_3	0	0	1	B

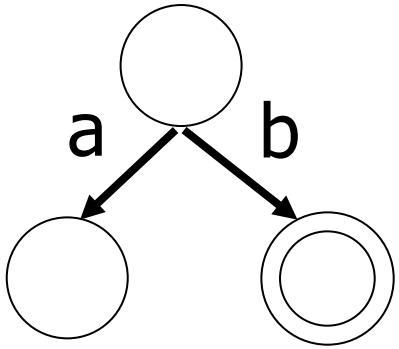
状態数最小化(step 1)

	F	a	b
q_0	○	q_0	q_1
q_1		q_1	q_2
q_2	○	q_2	q_3
q_3		q_3	q_0

	F	a	b	
q_0	1	1	0	A
q_1	0	0	1	B
q_2	1	1	0	A
q_3	0	0	1	B



A



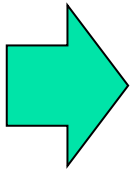
B

状態数最小化(step k)

- 各行のパターンを用いて，遷移表を書き換えてから各行のパターンを再分類

	F	a	b
q_0	○	q_0	q_1
q_1		q_1	q_2
q_2	○	q_2	q_3
q_3		q_3	q_0

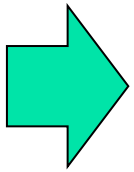
	F	a	b	
q_0	1	1	0	A
q_1	0	0	1	B
q_2	1	1	0	A
q_3	0	0	1	B



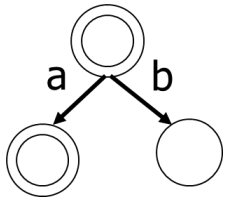
	F	a	b	
q_0	1	A	B	A’
q_1	0	B	A	B’
q_2	1	A	B	A’
q_3	0	B	A	B’

状態数最小化(step k)

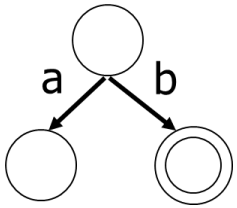
	F	a	b	
q_0	1	1	0	A
q_1	0	0	1	B
q_2	1	1	0	A
q_3	0	0	1	B



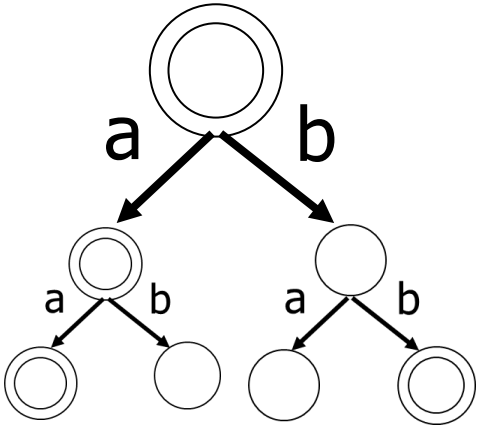
	F	a	b	
q_0	1	A	B	A’
q_1	0	B	A	B’
q_2	1	A	B	A’
q_3	0	B	A	B’



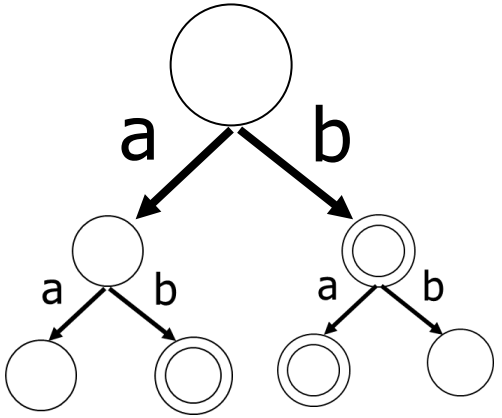
A



B



A’



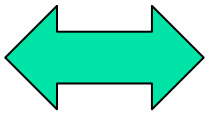
B’

状態数最小化(step k)

- 行の分類がもとの遷移表と変化がなければ終了.
  変化があれば，前頁へ戻る

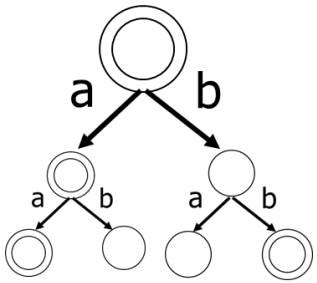
	F	a	b
q_0	○	q_0	q_1
q_1		q_1	q_2
q_2	○	q_2	q_3
q_3		q_3	q_0

	F	a	b	
q_0	1	1	0	A
q_1	0	0	1	B
q_2	1	1	0	A
q_3	0	0	1	B

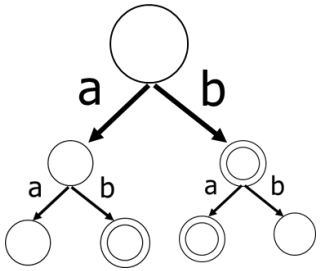


	F	a	b	
q_0	1	A	B	A’
q_1	0	B	A	B’
q_2	1	A	B	A’
q_3	0	B	A	B’

状態数最小化(手順の終了)

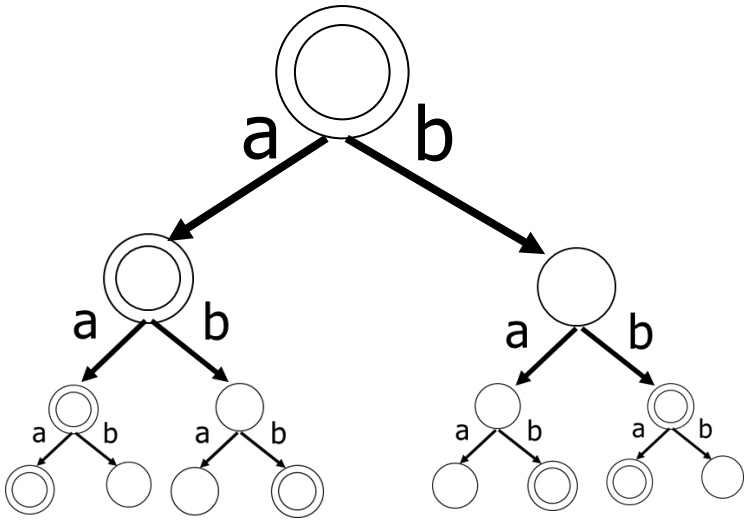


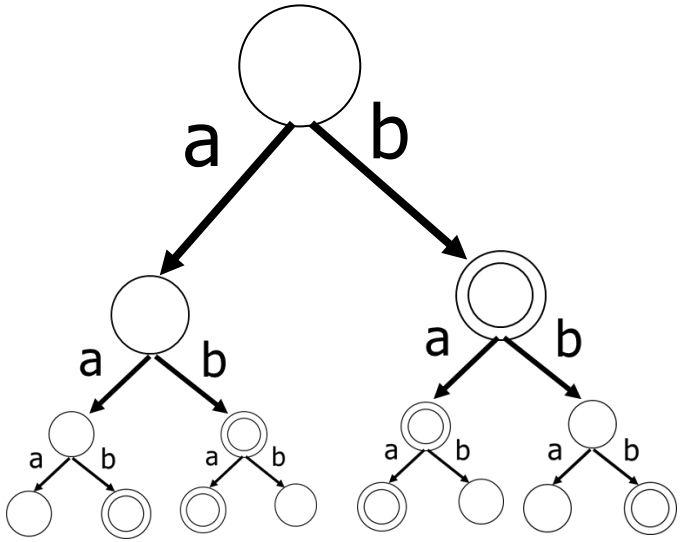
A′



B′

	F	a	b	
q_0	1	A	B	A’
q_1	0	B	A	B’
q_2	1	A	B	A’
q_3	0	B	A	B’



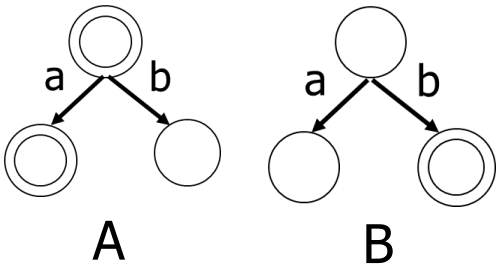


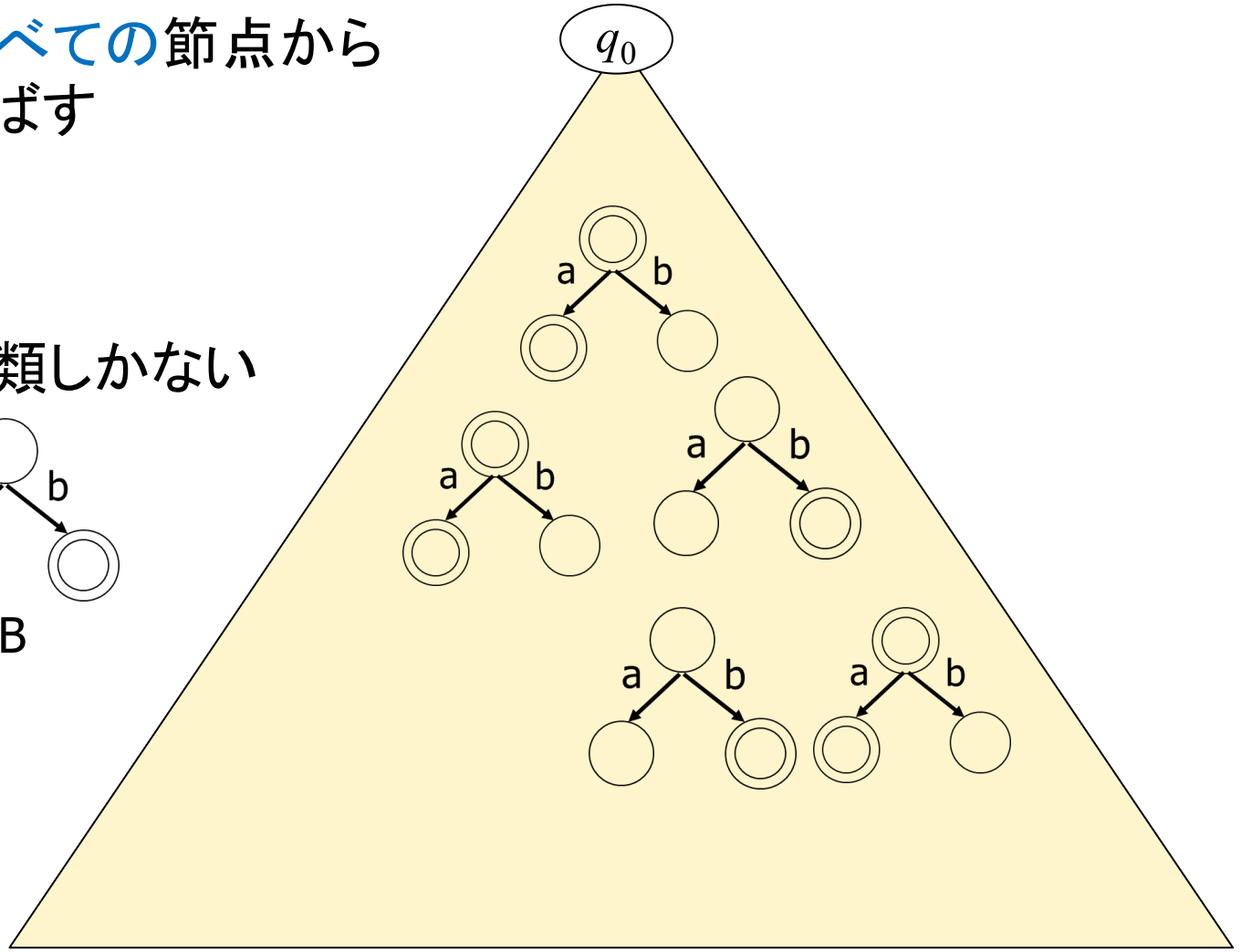
手順終了の根拠(1)

- 前綴木中のすべての節点から一斉に子を延ばす

状態遷移表より

親子関係は2種類しかない



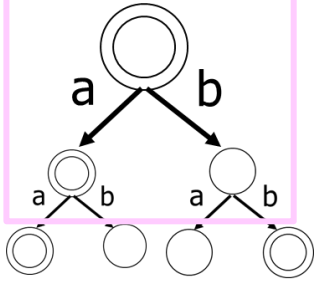


手順終了の根拠(2)

状態遷移表より

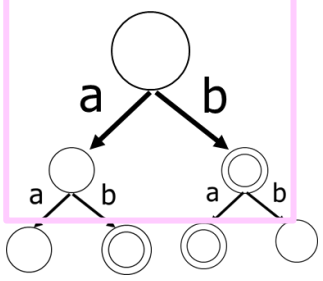
3世代関係も2種類しかない

A



A′

B

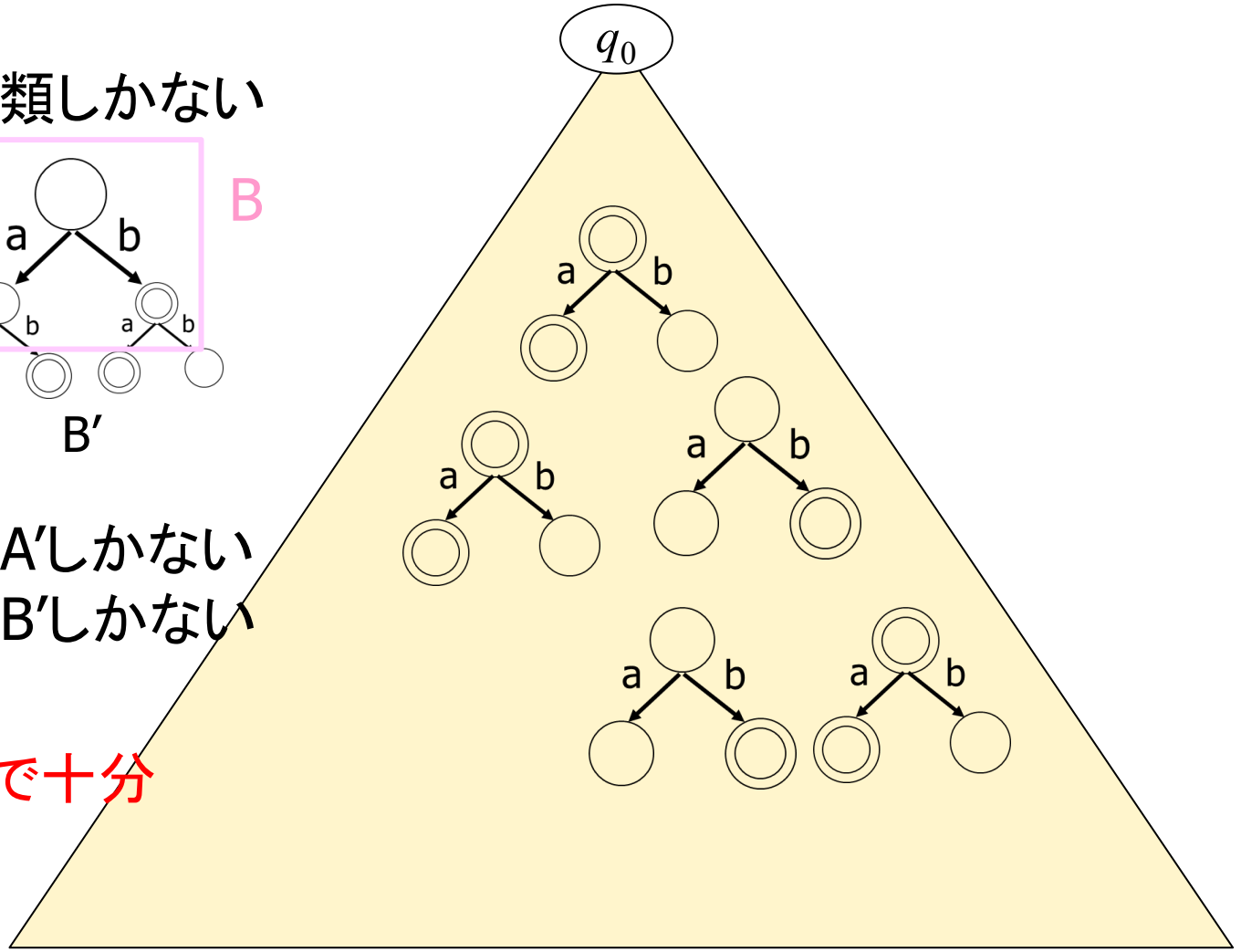


B′

Aから延長してもA′しかない

Bから延長してもB′しかない

状態は2つだけで十分



状態数最小化アルゴリズム

アルゴリズムの入力 $M = (\Sigma, S, \delta, q_0, F)$

アルゴリズムの出力 $M' \equiv M$ なる M' で状態数が最小のもの

※以下では $k = 0, 1, 2, ..$ に対して状態間の同値関係$\equiv_k$と同値類$C_{k0}, C_{k1}, ...$を次々と構成する. このとき, q_iが属する同値類を$C_k(q_i)$で表す.

※関数 $f: S \to \{0, 1\}$を $f(q_i)=1 \Leftrightarrow q_i \in F$ と定義する.

手順

1. M から到達不可能な状態をすべて除去する.
2. $C_{00}=F, C_{01}=S-F$とおく
3. k まで同値関係$\equiv_k$と同値類$C_{k0}, C_{k1}, ...$が定義されていたとするとき,

   $q_i \equiv_{k+1} q_j \Leftrightarrow$

   $f(q_i) = f(q_j)$ かつ すべての$c \in \Sigma$について$C_k(\delta(q_i, c)) = C_k(\delta(q_j, c))$

   と定義したときの同値類を$C_{(k+1)\ 0}, C_{(k+1)\ 1}, ...$とする.
4. もし 同値類の族として $\{C_{k0}, C_{k1}, ...\} = \{C_{(k+1)\ 0}, C_{(k+1)\ 1}, ...\}$であれば終了.

   異なっていればkを1増やして3に戻る

状態数最小化アルゴリズム(問)

問1 同値関係$\equiv_0$を定義せよ

問2 同値関係 $q_i \equiv_{k+1} q_j$ ならば $q_i \equiv_k q_j$ であること，つまり
同値関係$\equiv_{k+1}$は$\equiv_k$の細分であることを示せ．

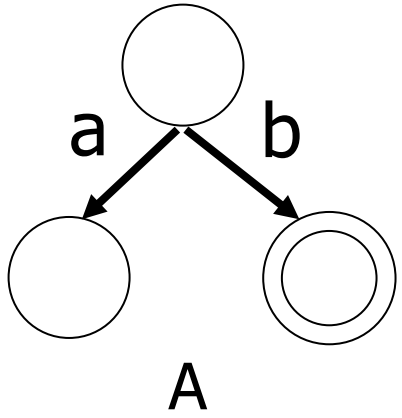
複雑な例

	F	a	b
q_0		q_1	q_2
q_1		q_4	q_7
q_2	○	q_2	q_0
q_3	○	q_2	q_1
q_4		q_2	q_6
q_5		q_4	q_7
q_6	○	q_5	q_1
q_7		q_5	q_6

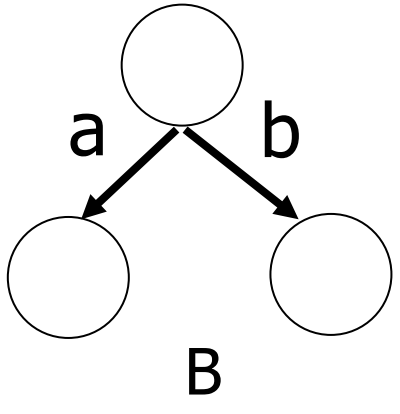
	F	a	b	
q_0	0	0	1	A
q_1	0	0	0	B
q_2	1	1	0	C
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	1	1	D
q_5	0	0	0	B
q_6	1	0	0	E
q_7	0	0	1	A

複雑な例

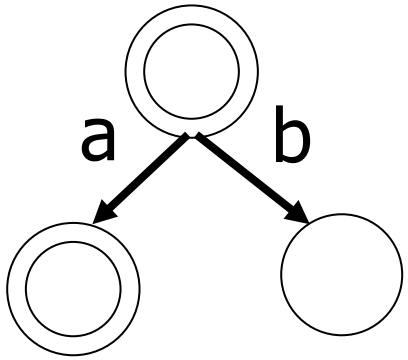
	F	a	b	
q_0	0	0	1	A
q_1	0	0	0	B
q_2	1	1	0	C
~~q_3~~	~~0~~	~~q_2~~	~~q_1~~	
q_4	0	1	1	D
q_5	0	0	0	B
q_6	1	0	0	E
q_7	0	0	1	A



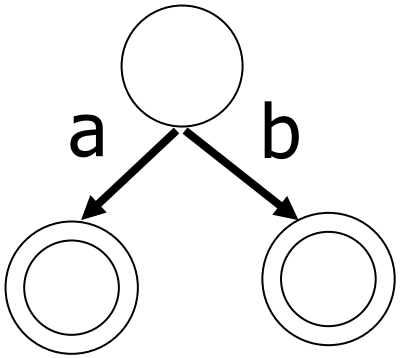
A



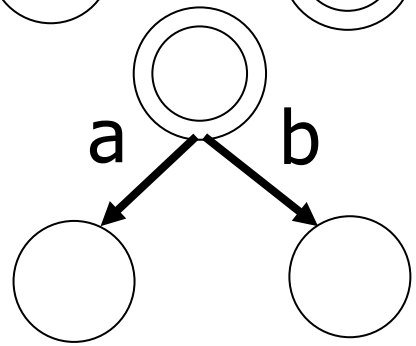
B



C



D



E

複雑な例(2)

	F	a	b
q_0		q_1	q_2
q_1		q_4	q_7
q_2	○	q_2	q_0
q_3	○	q_2	q_1
q_4		q_2	q_6
q_5		q_4	q_7
q_6	○	q_5	q_1
q_7		q_5	q_6

	F	a	b	
q_0	0	0	1	A
q_1	0	0	0	B
q_2	1	1	0	C
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	1	1	D
q_5	0	0	0	B
q_6	1	0	0	E
q_7	0	0	1	A

	F	a	b	
q_0	0	B	C	A’
q_1	0	D	A	B’
q_2	1	C	A	C’
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	C	E	D’
q_5	0	D	A	B’
q_6	1	B	B	E’
q_7	0	B	E	F’

複雑な例(2)

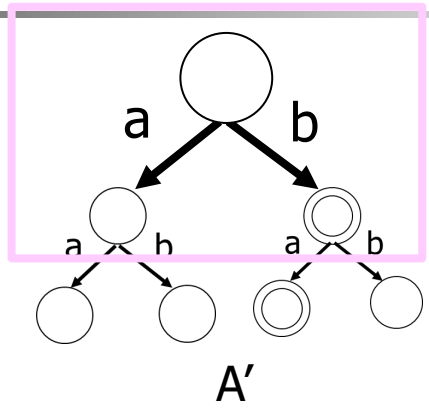
	F	a	b
q_0		q_1	q_2
q_1		q_4	q_7
q_2	○	q_2	q_0
q_3	○	q_2	q_1
q_4		q_2	q_6
q_5		q_4	q_7
q_6	○	q_5	q_1
q_7		q_5	q_6

	F	a	b	
q_0	0	0	1	A
q_1	0	0	0	B
q_2	1	1	0	C
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	1	1	D
q_5	0	0	0	B
q_6	1	0	0	E
q_7	0	0	1	A

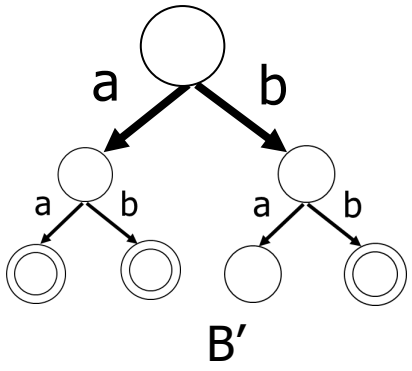
	F	a	b	
q_0	0	B	C	A’
q_1	0	D	A	B’
q_2	1	C	A	C’
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	C	E	D’
q_5	0	D	A	B’
q_6	1	B	B	E’
q_7	0	B	E	F’

複雑な例(2)

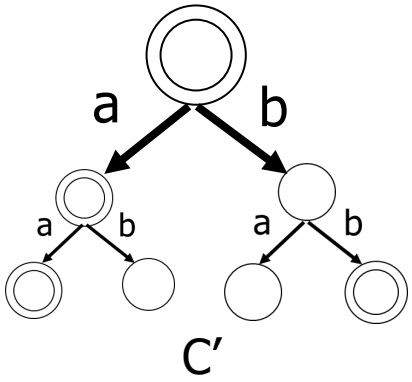
	F	a	b	
q_0	0	B	C	A’
q_1	0	D	A	B’
q_2	1	C	A	C’
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	C	E	D’
q_5	0	D	A	B’
q_6	1	B	B	E’
q_7	0	B	E	F’



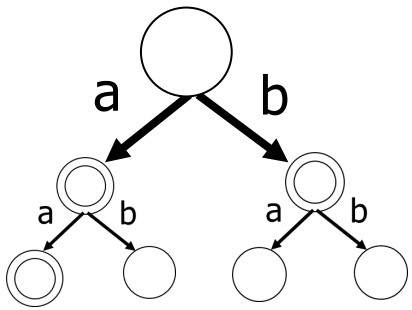
A′



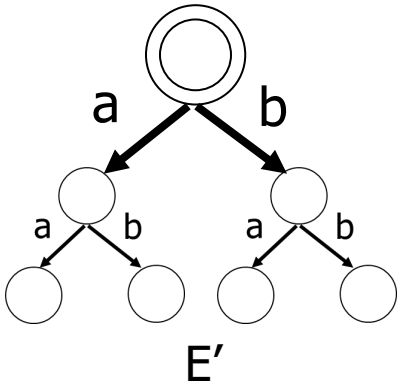
B′



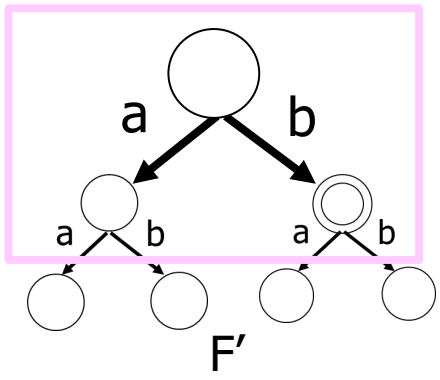
C′



D′



E′



F′

複雑な例(2)

	F	a	b
q_0		q_1	q_2
q_1		q_4	q_7
q_2	○	q_2	q_0
q_3	○	q_2	q_1
q_4		q_2	q_6
q_5		q_4	q_7
q_6	○	q_5	q_1
q_7		q_5	q_6

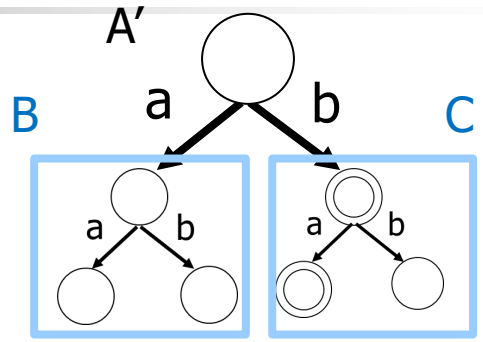
	F	a	b	
q_0	0	B	C	A’
q_1	0	D	A	B’
q_2	1	C	A	C’
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	C	E	D’
q_5	0	D	A	B’
q_6	1	B	B	E’
q_7	0	B	E	F’

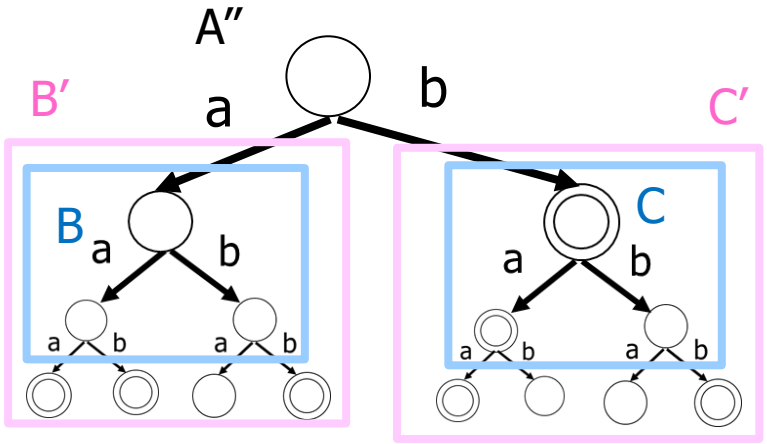
	F	a	b	
q_0	0	B’	C’	A”
q_1	0	D’	F’	B”
q_2	1	C’	A’	C”
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	C’	E’	D”
q_5	0	D’	F’	B”
q_6	1	B’	B’	E”
q_7	0	B’	E’	F”

複雑な例(2)

	F	a	b	
q_0	0	B	C	A’
q_1	0	D	A	B’
q_2	1	C	A	C’
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	C	E	D’
q_5	0	D	A	B’
q_6	1	B	B	E’
q_7	0	B	E	F’

	F	a	b	
q_0	0	B’	C’	A”
q_1	0	D’	F’	B”
q_2	1	C’	A’	C”
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	C’	E’	D”
q_5	0	D’	F’	B”
q_6	1	B’	B’	E”
q_7	0	B’	E’	F”

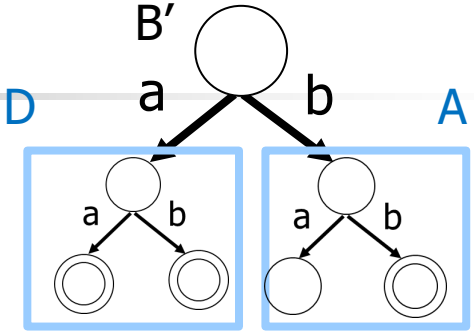


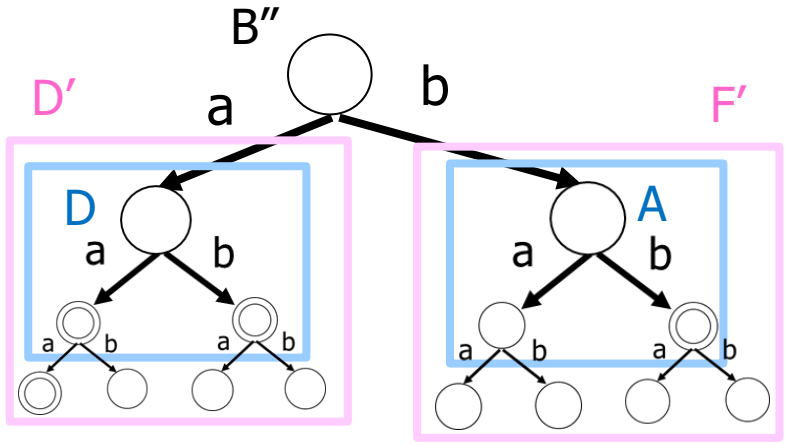


手順終了の根拠

	F	a	b	
q_0	0	B	C	A’
q_1	0	D	A	B’
q_2	1	C	A	C’
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	C	E	D’
q_5	0	D	A	B’
q_6	1	B	B	E’
q_7	0	B	E	F’

	F	a	b	
q_0	0	B’	C’	A”
q_1	0	D’	F’	B”
q_2	1	C’	A’	C”
~~q_3~~	~~○~~	~~q_2~~	~~q_1~~	
q_4	0	C’	E’	D”
q_5	0	D’	F’	B”
q_6	1	B’	B’	E”
q_7	0	B’	E’	F”

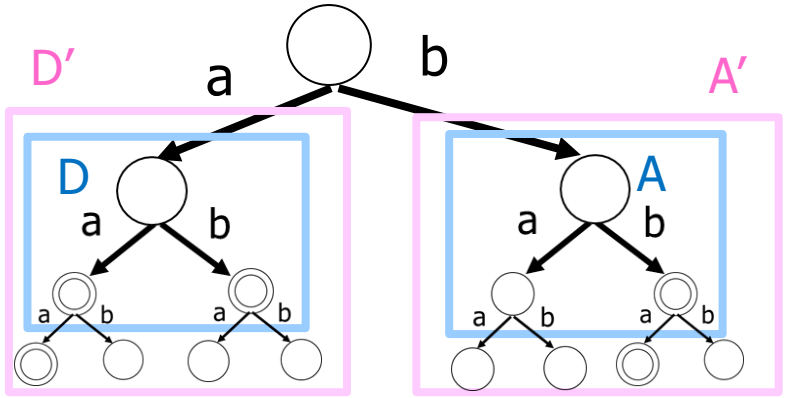




状態遷移表に現れない



B”が2つの状態に分解されることはない



到達不可能な状態を除去しないと…

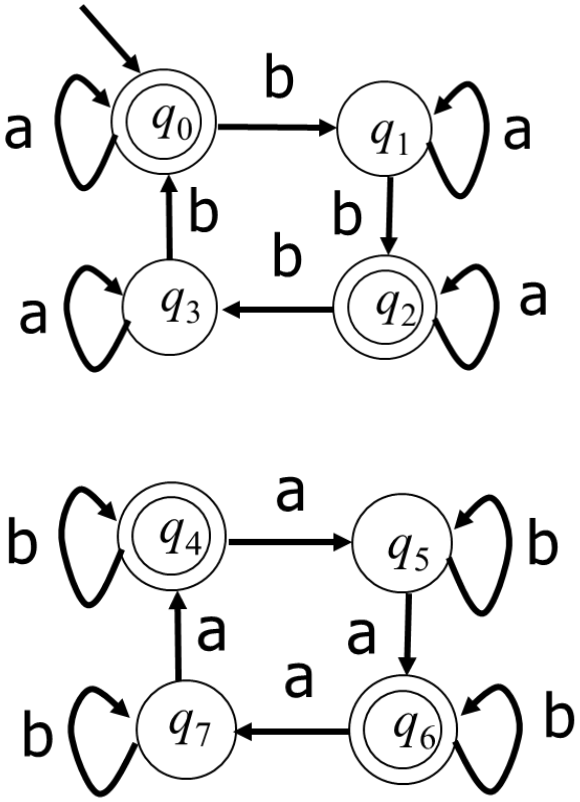
	F	a	b
q_0	○	q_0	q_1
q_1		q_1	q_2
q_2	○	q_2	q_3
q_3		q_3	q_0
q_4	○	q_5	q_4
q_5		q_6	q_5
q_6	○	q_7	q_6
q_7		q_4	q_7

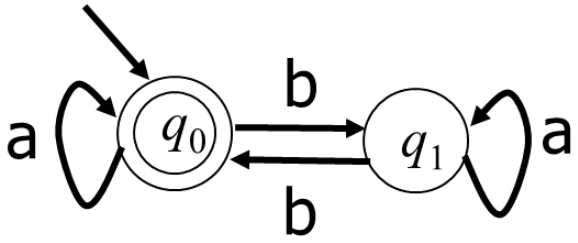
	F	a	b	
q_0	1	1	0	A
q_1	0	0	1	B
q_2	1	1	0	A
q_3	0	0	1	B
q_4	1	0	1	C
q_5	0	1	0	D
q_6	1	0	1	C
q_7	0	1	0	D

	F	a	b	
q_0	0	A	B	A’
q_1	0	B	A	B’
q_2	1	A	B	A’
q_3	0	B	A	B’
q_4	1	D	C	C’
q_5	0	C	D	D’
q_6	1	D	C	C’
q_7	0	C	D	D’

到達不可能な状態を除去しないと...

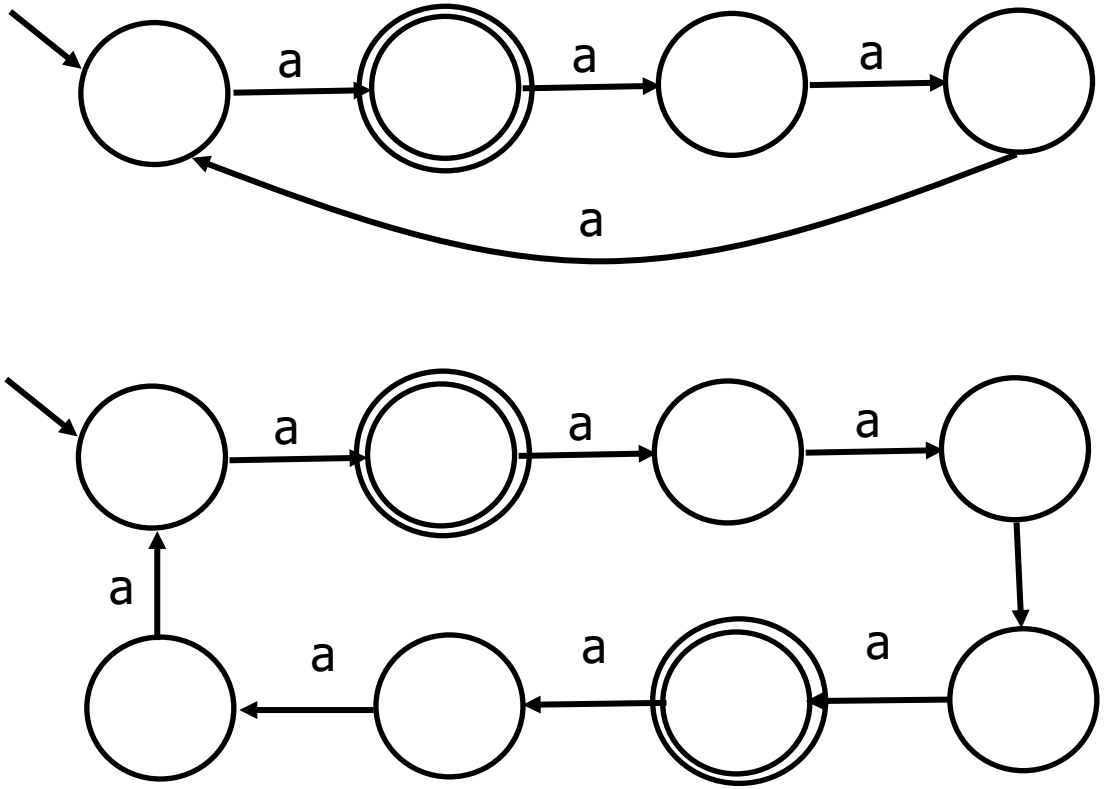
	F	a	b
q_0	○	q_0	q_1
q_1		q_1	q_2
q_2	○	q_2	q_3
q_3		q_3	q_0
q_4	○	q_5	q_4
q_5		q_6	q_5
q_6	○	q_7	q_6
q_7		q_4	q_7





有限状態機械の例(1)より

- $L=\{ a^n \mid n = 4m+1, m \in \mathbf{N}\}$を受理する有限状態オートマトン



状態数以上の語に対する動作

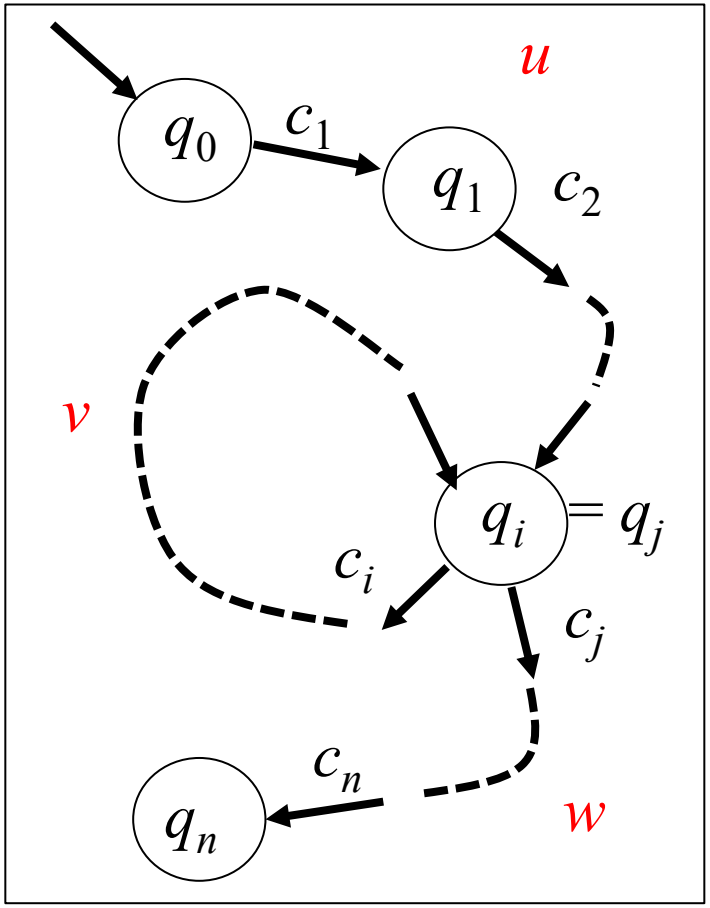
補題 有限状態オートマトンMの状態数をnとするとき, 長さnの語$s = c_1 c_2 \cdots c_n$をMの入力として与えて

$$q_0 \overset{c_1}{\Rightarrow} q_1 \overset{c_2}{\Rightarrow} q_2 \overset{c_3}{\Rightarrow} \cdots \overset{c_n}{\Rightarrow} q_n$$

と遷移したとすると, $q_i = q_j$ $(i \neq j)$ なる組が存在する

- 鳩ノ巣原理による.

$\delta(q_0, c_1) = q_1, \delta(q_1, c_2) = q_2, \cdots$ という意味



系 補題においてsの部分語uwで$|uw| < |s|$かつ, 一度も同じ状態に遷移せずにq_nに到達するものを構成することができる.

正則言語の特徴

Pumping Lemma 要素数が有限ではない正則言語Lに対して, 自然数Nがあって, $s \in L$かつ$|s| \geq N$ なるすべての語sは $s=uvw$ かつ $|v| \geq 1$かつ すべての$i \geq 0$に対して $uv^iw \in L$ が成り立つような部分語vを含んでいる.

- NはLに依存して定まる(Lが変わると Nは変わる)
- Nは$L=L(M)$なる有限状態オートマトン Mの状態数+1とすればよい.

(注) 自然数の除法の基本式 $b = qa+r$ と類似している

Myhill-Nerodeの定理

Theorem 次の3条件は同値である:
(1) 文字列の集合 Lはある有限オートマトン M で受理される.
(2) L は, Σ^*を右不変で有限指数のある同値関係によって分割したときの, 同値類の和集合になっている.
(3) Σ^*上の同値関係 R_L を

$$x\, R_L\, y \quad \Leftrightarrow \quad \text{“for all } z \in \Sigma^* \;\; xz \text{ is in } L \text{ iff } yz \text{ is in } L\text{”}$$

と定義すると, R_L は有限指数である.

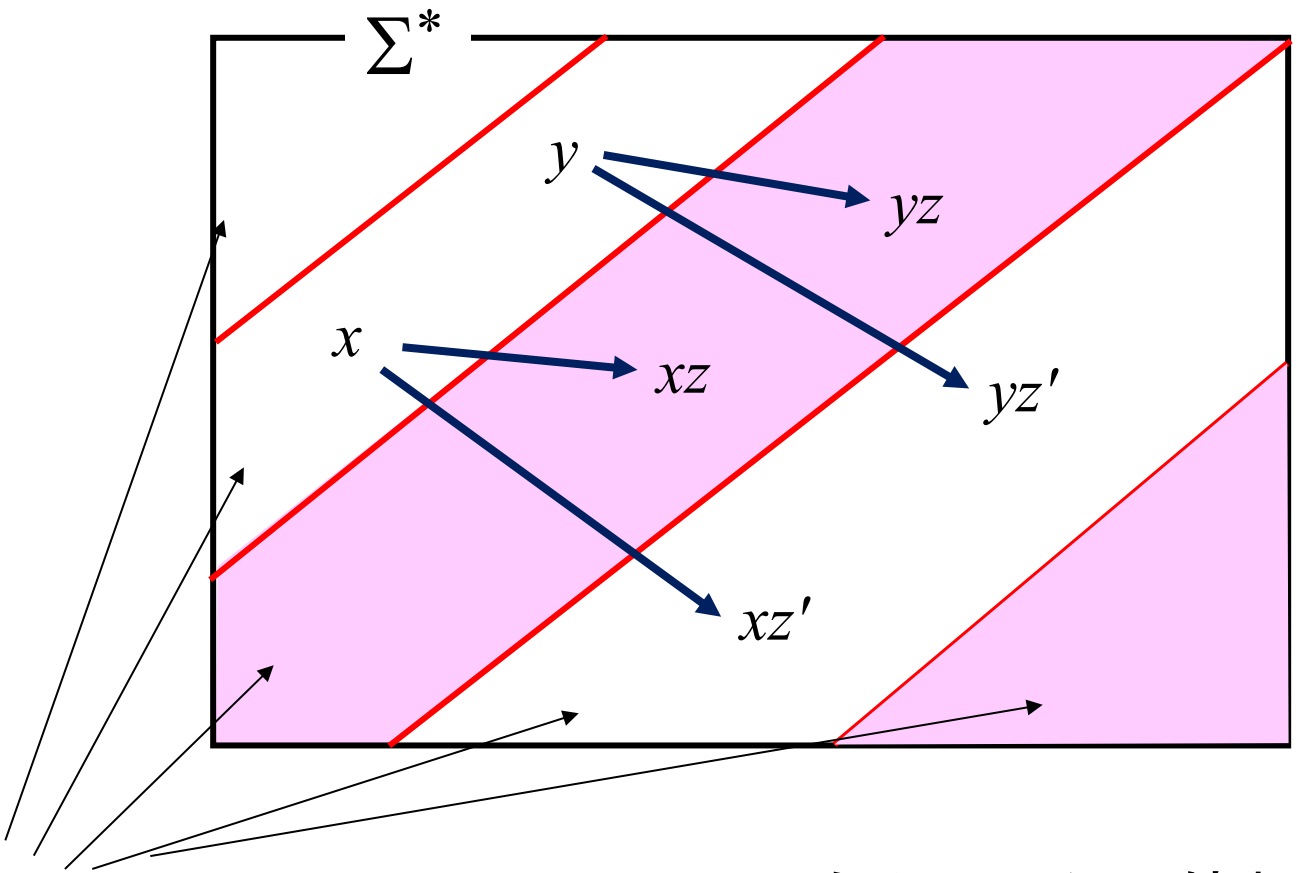
- 同値関係Rが右不変である(right invariant) とは

  $$x\, R\, y \text{ implies } xz\, R\, yz \text{ for all } z \in \Sigma^*$$

  を満たすことをいう.
- 同値関係R で分割された同値類の数をRの指数(index)と呼ぶ.

- 条件(3)のR_Lを用いるとLを受理する状態数最小の有限オートマトンが構成できる.

Myhill-Nerodeの定理の図解



各同値類がオートマトンの状態に対応する.

色付けした同値類が受理状態に対応する.

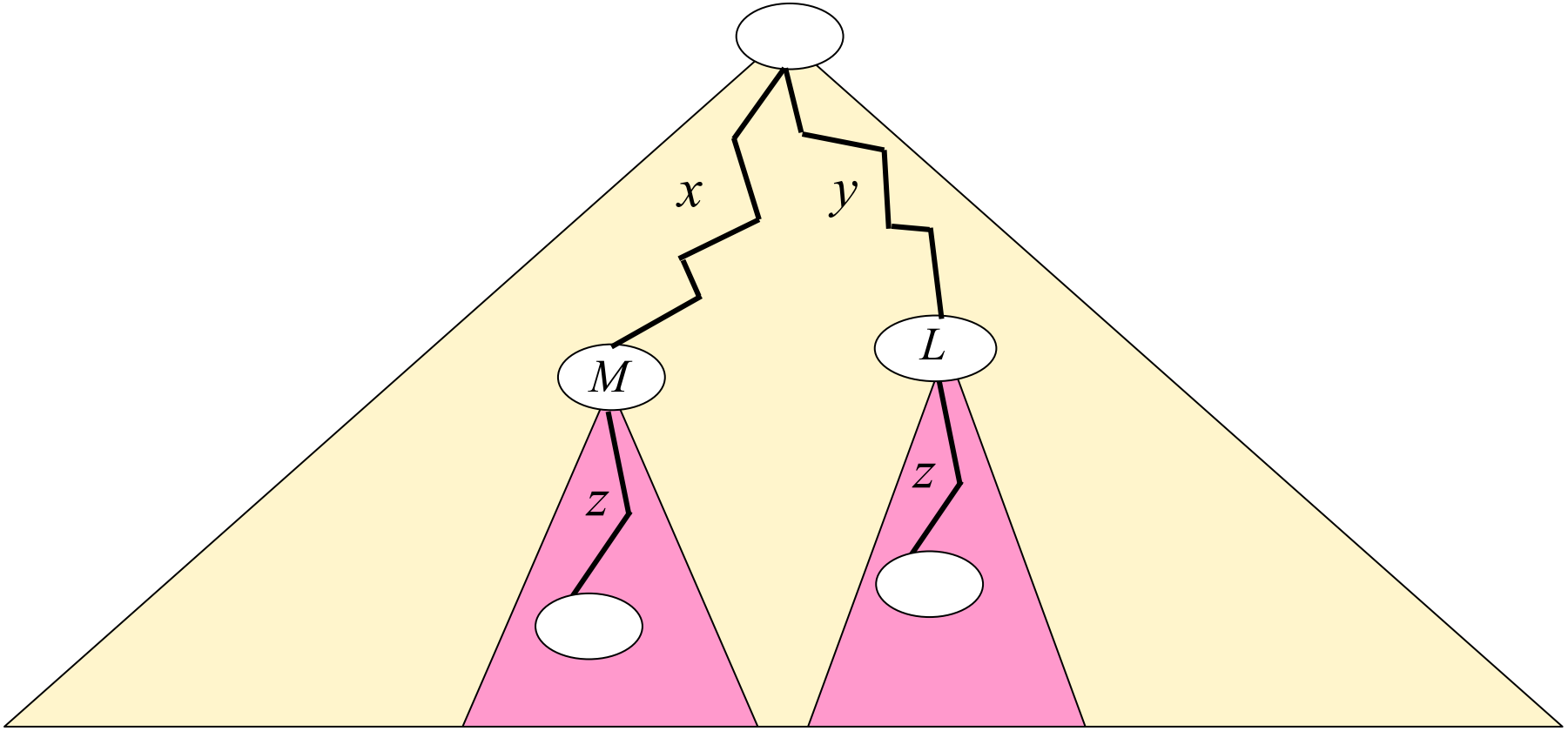
状態遷移とは同値類遷移である

条件3の図解

(3) Σ^*上の同値値関係 R_L を

$x\, R_L\, y \quad \Leftrightarrow \quad$ “for all $z \in \Sigma^*$ xz is in L iff yz is in L”

と定義すると, R_L は有限指数である.



証明 (1)⇒(2)

証明の前に準備

言語Lを受理するオートマトンを$M = (\Sigma, S, \delta, q_0, F)$とする.

文字列$w = c_1 c_2 \cdots c_n$列に対して,

$$\delta(q_0, c_1) = q_1, \delta(q_1, c_2) = q_2 , ..., \delta(q_{n-1}, c_n) = q_n$$

$$q_0 \overset{c_1}{\Rightarrow} q_1 \overset{c_2}{\Rightarrow} q_2 \overset{c_3}{\Rightarrow} \cdots \overset{c_n}{\Rightarrow} q_n$$

のとき, $q_n = \delta(w) = \delta(c_1 c_2 \cdots c_n)$ と表記する. つまり関数δ を

$$\delta: \Sigma^* \to S$$

であるような関数と定義しなおす.

証明 (1)⇒(2) 続き

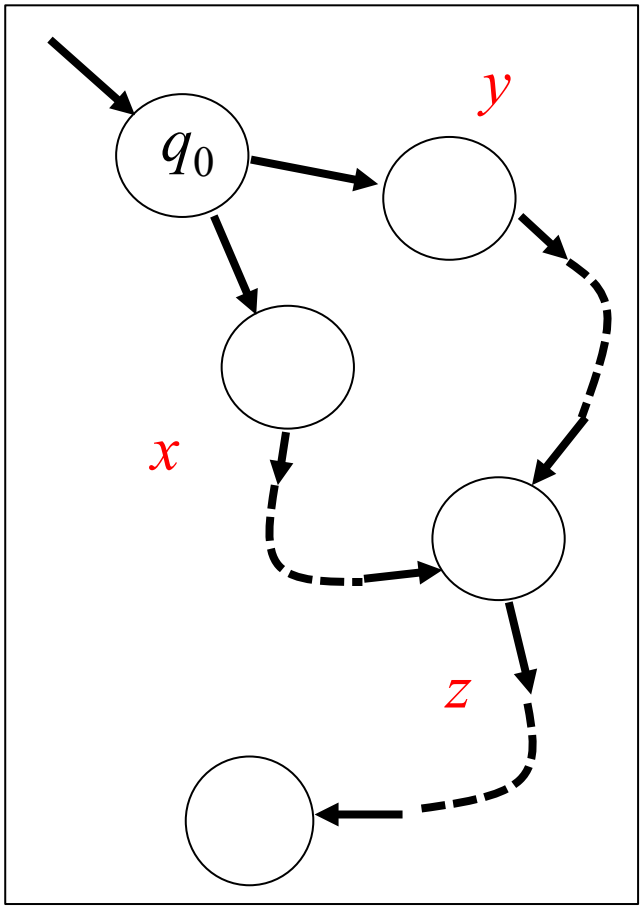
- M を用いて, Σ^* 上の関係 R_M を

$$x\, R_M y \quad \Leftrightarrow \quad \delta(x) = \delta(y)$$

と定義する. この関係が反射律, 対称律, 推移律を満たすことは容易に確かめることができる. さらに,関数$\delta:\Sigma^* \to S$ の定義から直ちに

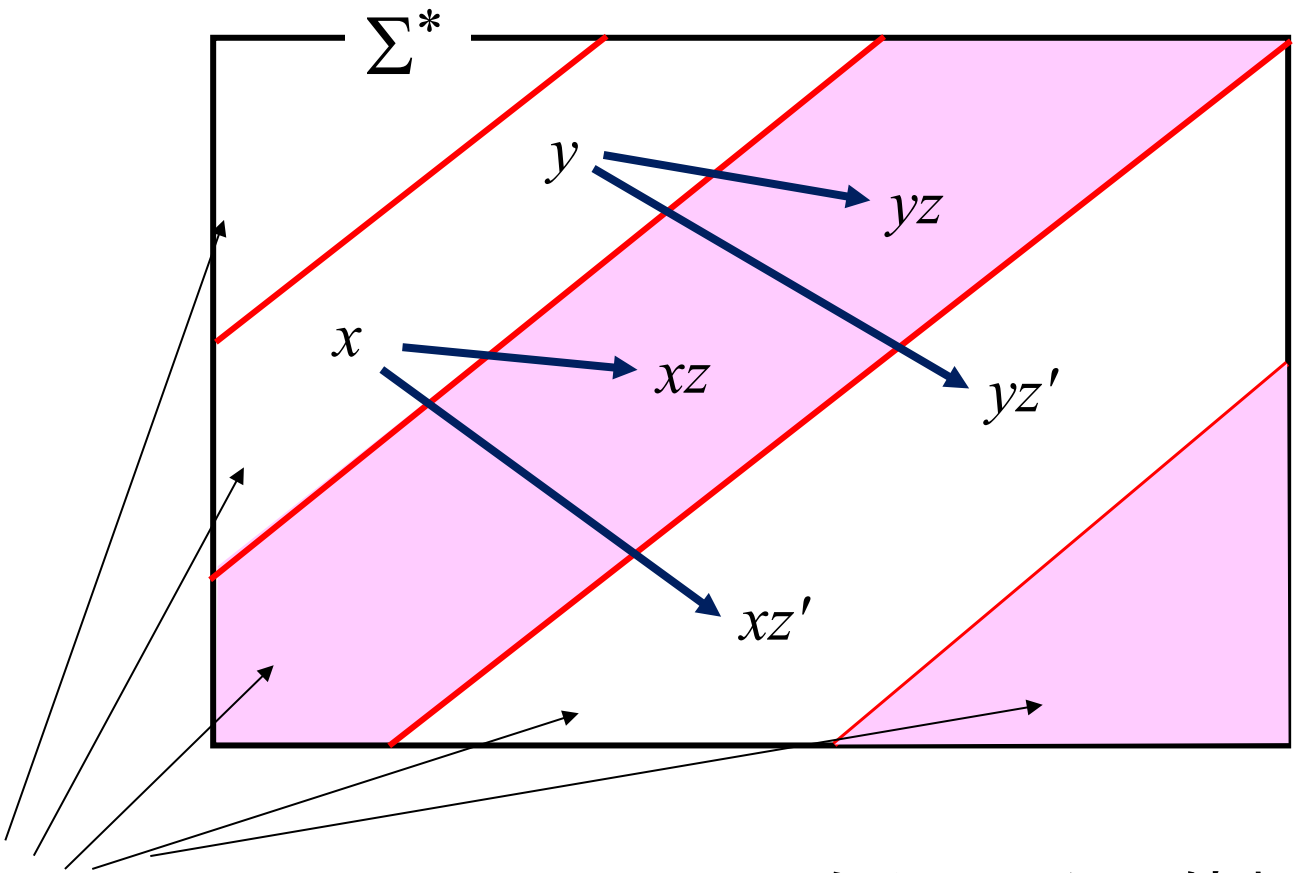
任意の$z \in \Sigma^*$に対して, $\delta(x) = \delta(y)$であれば$\delta(xz) = \delta(yz)$

が成立する.



- Mの状態数 $|S|$ は有限であるから, R_M の同値類の数は有限である.
- $\delta(w) \in F$ であるような$[w]$の和集合がLになる. (このような同値類も有限個しかない).

Myhill-Nerodeの定理の図解



各同値類がオートマトンの状態に対応する.

色付けした同値類が受理状態に対応する.

状態遷移とは同値類遷移である

証明 (2)⇒(3)

最初に「有限指数」を除いて議論する

- 言語LがΣ^*を右不変な同値関係Rによって分割したときの，同値類の和集合になっていると仮定する．つまり

  $$x\,R\,y \quad \Rightarrow \quad \text{for all } z\in\Sigma^* \;\; xz\,R\,yz$$

  と仮定する．
- $x\,R\,y$ が成立していると仮定し，$z\in\Sigma^*$を任意に固定する．
- これだけの仮定のもとで

  $$xz \text{ is in } L \text{ iff } yz \text{ is in } L \qquad (*)$$

  を示す．

$xz\in L$ と仮定すると，右不変性より

$xz\,R\,yz$，つまり$[xz]=[yz]$．

LはRの同値類の和集合であったのだから，$yz\in L$である．

逆に$yz\in L$ と仮定しても$xz\in L$ 導くことができる．

したがって$(*)$が成り立つ．

証明 (2)⇒(3) 続き

- $z \in \Sigma^*$は任意に固定したので

$$x\,R\,y \text{ ならば } \text{“for all } z \in \Sigma^* \ xz \text{ is in } L \text{ iff } yz \text{ is in } L\text{”}$$

すなわち $x\,R\,y$ ならば $x\,R_L\,y$ が成り立つ.

- ここで「ならば」の後の命題を用いて Σ^*上の同値関係 R_L を

$$x\,R_L\,y \quad \Leftrightarrow \quad \text{“for all } z \in \Sigma^* \ xz \text{ is in } L \text{ iff } yz \text{ is in } L\text{”}$$

と定義すると

$$x\,R\,y \text{ ならば } x\,R_L\,y.$$

- RとR_Lともに同値関係であり, R_L による分割はRによる分割より粗いことが示せた.
- R_Lによる同値類の個数はRによる同値類の個数を超えることはない. したがって, Rが有限指数ならば, R_Lも有限指数である.

(注1) R_LはLによって決まること, (1) ⇒ (2) の証明で作ったR_Mによる同値類の個数はR_Lによる同値類の個数を超えない.

証明 (3)⇒(1)

- 同値関係 $x\ R_L\ y \Leftrightarrow$“for all $z \in \Sigma^*$ xz is in L iff yz is in L”が有限指数であるとき, Lを受理する有限オートマトン$M' = (\Sigma, S', \delta', q'_0, F')$を

$$S' = \{ [x] \mid x \in \Sigma^* \},\ q'_0 = [\varepsilon],\ F' = \{ [x] \mid x \in L \}$$

と定義する.

- 遷移関数δ'については

$$\delta'([x], c) = [xc]$$

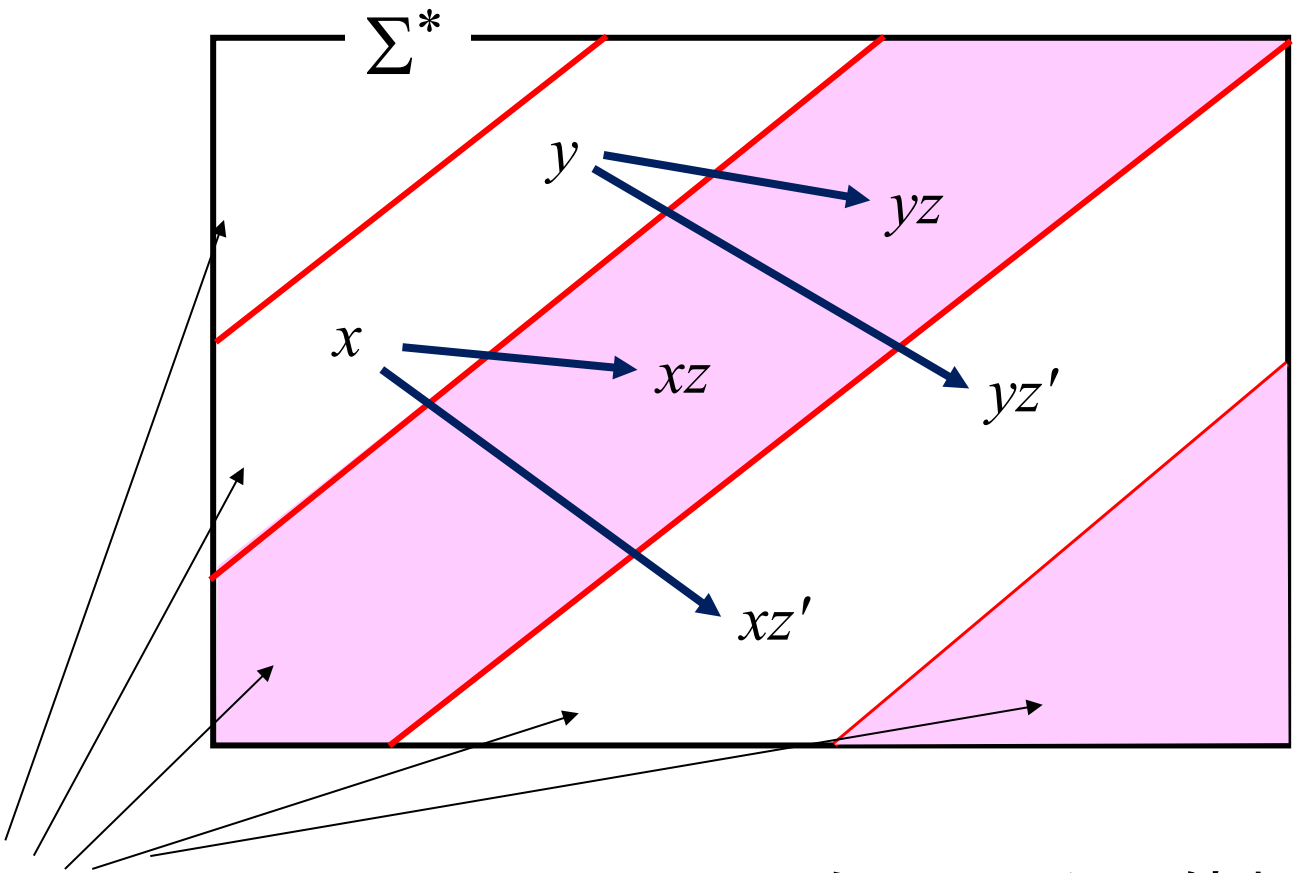
と定義したい.

- この定義が矛盾なく定義されている(well-defined)ことを示すために

$[x] = [y]$なる任意の y に対して $[xc] = [yc]$

であることを示す.

Myhill-Nerodeの定理の図解



各同値類がオートマトンの状態に対応する.

色付けした同値類が受理状態に対応する.

状態遷移とは同値類遷移である

証明 (3)⇒(1)

(3) Σ^*上の同値関係 R_L を

$x\, R_L\, y \quad \Leftrightarrow$ “for all $z \in \Sigma^*$ xz is in L iff yz is in L”

と定義すると, R_L は有限指数である.

目標: $[x]=[y]$なる任意の y に対して $[xc]=[yc]$であること

- $[x]=[y]$とは xとyが同じ同値類に入っている, つまり, $x\, R_L\, y$.
- (3)より, 任意の$z \in \Sigma^*$について $xz \in L$ iff $yz \in L$ だから,

  $|z| \geq 1$ のときに限定すれば, $z = cw$とおいて,

  任意の$w \in \Sigma^*$ に対して$xcw \in L$ iff $ycw \in L$

つまり, $xc\, R_L\, yc$が成り立つ. よって

$[x]=[y]$ならば$[xc]=[yc]$.

$L(M')=L$であることはM'の定義から直接導かれる.

(注2) (注1)より M'がLを受理する有限オートマトンで状態数が最小である.